



University of Stuttgart

Institute for Control Engineering of Machine
Tools and Manufacturing Units (ISW)

Bachelor thesis

VHDL and embedded algorithms on an FPGA/NIOS CPU for distributed controller synchronization and motion control execution

A Bachelor thesis presented to the faculty Production and Automotive
Engineering, University of Stuttgart, in partial fulfillment of the requirements for
the degree of Bachelor of Engineering.

submitted by

Ahmed Mahfouz

from Cairo, Egypt

Examiner: Prof. Dr.-Ing Alexander Verl

Supervisor: Caren Dripke, M.Sc.

Submitted on: 09.August 2020

Institute for Control Engineering of Machine Tools and Manufacturing Units (ISW),
University of Stuttgart

August 2020

Declaration of independence

Student research project of Mr. Ahmed Mahfouz (Mechatronics)

Address	Allmandring 24B, 70569 Stuttgart
Home address	Cairo, Egypt
Registration number	3500154
German title	VHDL und embedded Algorithmen auf einer FPGA/NIOS-CPU für verteilte Steuerungssynchronisation und Ausführung der Bewegungssteuerung
English title	VHDL and embedded algorithms on an FPGA/NIOS CPU for distributed controller synchronization and motion control execution

I hereby declare,

- that I wrote this paper independently,
- that no sources other than those indicated are used and that all statements taken literally or in spirit from other works are marked as such,
- that the submitted work has not been the subject of another examination procedure, neither in its entirety nor in substantial parts,
- that I have not yet published the work, either in whole or in part,
- that my work does not violate the rights of third parties and that I release the university from any claims of third parties.

Abstract

The purpose of this thesis is to integrate and synchronize an embedded NIOS CPU called the Effectuator into the Simulink model of a three-dimensional axes hardware system called demonstrator. The thesis reviewed the previous work related to this project. The integration process started with an implementation of **UDP** communication for both the Effectuator and the demonstrator. With the communication implementation, the Effectuator was inserted into the common working area of the demonstrator, where all axes can share their coordination data. After that, motion control algorithms were implemented to actuate X2-axis. This axis will be driven by the Effectuator to superimpose the motion of the X-axis of the demonstrator. As a result, the demonstrator was extended by a new transitional axis, and an agent integration criteria were implanted and tested so more agents can be integrated with the same procedure.

Keywords : **UDP** communication, embedded systems, decentralization, synchronization, motion control.

Acknowledgement

I wish to express my sincere appreciation to my supervisor, **Caren Dripke, M.Sc.** from whom I learned a lot of useful skills in both technical and academic aspects. Honestly, she was one of the most organized, dedicated, and supportive persons I have ever met who helped me much to do my bachelor thesis easily and smoothly.

The physical and educational contribution of the ‘**German University in Cairo**’ is truly appreciated. Without their support and funding, I would not have the opportunity to do my bachelor thesis abroad at one of the oldest technical universities in Germany, as ‘**Stuttgart University**’.

I wish to thank all the people whose assistance was a milestone in the completion of this project:

- **Dipl.-Ing. Mihai Dragan**, whose assistance had a great effect on the thesis implementation by getting the simulation laptop supported with the admin rights.
- **Mr. Georg Ziegler**, who supported me with the technical materials during my stay at the **ISW** Application lab.

I wish to acknowledge the support and great love of my family, my father, **Mahmoud**; my mother, **Amal**; my brother, **Muhammad**; and my sister **Eman**. They kept me going on, and this work would not have been possible without their support and encouragement.

Contents

Declaration of independence	i
Acknowledgement	iii
List of figures	viii
List of tables	ix
List of Abbreviations	x
1 Introduction	1
1.1 Motivation	1
1.2 Aim of thesis	2
1.3 Requirements of thesis	3
2 Analysis of previous work on DEVEKOS research project	4
2.1 Fundamental concepts	4
2.1.1 Skill based-axes concept	4
2.1.2 Holonic Manufacturing Control	5
2.1.3 Decentralization concept	6
2.1.4 Distributed motion controller concept	6
2.1.5 Interpolation concept	7
2.1.6 Implementation challenges	8
2.2 Initial setup	8
2.2.1 The demonstrator	8
2.2.2 The Simulink model	9
2.2.3 The Effectuator	11
2.3 The system implemented by the thesis	11

3	Communication between Effectuator and the original Simulink model	13
3.1	Industrial communication background	13
3.2	Analysis of previous work on the communication task	15
3.2.1	The original Simulink model	15
3.2.2	The Effectuator	16
3.3	Comparison between TCP and UDP	17
3.4	TCP and UDP comparison conclusion	20
3.5	Lightweight IP stack (lwIP)	21
3.6	Hardware setup	21
3.7	Software setup	24
3.8	Communication task implementation	26
3.8.1	Effectuator communication implementation	26
3.8.2	Simulink communication implementation	32
3.9	Implementation test using sent and received sine waves	34
3.9.1	Test results	35
3.9.2	Comments on results	35
3.9.3	Communication testing analysis	37
3.10	Summary of the chapter	38
4	Effectuator integration with the original Simulink model	39
4.1	Analysis of previous work on new agents integration	39
4.2	Incoming and out-coming UDP packets processing	40
4.3	Effectuator insertion into the original common working area	46
4.3.1	Simulink implementation	46
4.3.2	Effectuator implementation	48
4.4	Effectuator Simulation with a Simulink model	50
4.5	Summary of the chapter	54
5	Motion control for the new Effectuator axis	55
5.1	State of the art for motion control	55
5.1.1	PID controller	55
5.2	Analysis of previous work on axes motion control	56
5.3	Segment controller implementation	58
5.4	Axis control loop	60
5.4.1	PID controller implementation	62
5.4.2	Saturation block implementation	62
5.4.3	Discrete transfer function implementation	64

5.4.4	Discrete time integrator implementation	64
5.4.5	Control loop implementation	65
5.4.6	Go to position implementation	67
5.5	The Effectuator synchronization with the demonstrator axes .	69
5.6	Segment controller and control loop implementation debugging	71
5.7	Summary of the chapter	73
6	Evaluation	74
6.1	Evaluation of thesis requirements	74
6.2	The Effectuator and demonstrator synchronization analysis . .	75
6.3	The requirements for the real motor and non-simulated demonstrator	76
7	Summary and Outlook	78
7.1	Summary	78
7.2	Outlook	80
	Bibliography	83
A	Mathematical derivation for difference equations	84
A.1	Discrete PID controller	84
A.2	Discrete axis transfer equation	86

List of Figures

2.1	Cooperation types of a multi-axis system	5
2.2	Notation for set point segments S_m	7
2.3	The demonstrator existing at ISW	9
2.4	The input curve tracked by the transitional axes	10
2.5	The entire process of the new system	12
3.1	Communication Software structure	14
3.2	Comparison between TCP and UDP client program flow . .	19
3.3	Comparison between TCP and UDP server program flow . .	20
3.4	Hardware setup	22
3.5	NIOS II debug perspective	24
3.6	Wireshark Network Analyzer	25
3.7	PuTTY terminal	25
3.8	The Effectuator Free RTOS implementation	26
3.9	The Effectuator communication implementation	27
3.10	UDP _communication Simulink block	33
3.11	UDP_send Simulink block	33
3.12	UDP_receive Simulink block	34
3.13	communicationTesting Simulink model	34
3.14	Number of received frames for test cases 1, 3, and 5	35
4.1	Get_coordination_vector function behaviour	41
4.2	Get_consensus_vector function behaviour	42
4.3	Detect_motion_start/end function behaviour	42
4.4	Send_axes_index/position function behaviour	43
4.5	Output_selection function behaviour	44
4.6	Data type conversion diagram	45
4.7	Axis_X2_Effectuator flowchart	47

4.8	The curve traced by the Effectuator axis	48
4.9	Effectuator Simulink model	50
4.10	Generate_coordination_vector function behaviour	51
4.11	Generate_consensus_vector function behaviour	52
4.12	Generate_execution_vector function behaviour	52
4.13	Send_Effectuator_index/position function behaviour	53
5.1	PID Controller Feedback System	56
5.2	Discrete cascaded axis control loop	57
5.3	Segment controller behaviour	61
5.4	sat_Speed() function behaviour	63
5.5	Go_to_pos() function behaviour	68
5.6	The new handled case for maximum speed saturation block	70
5.7	Output position of the four axes	70
5.8	controlLoopDepugging Simulink model	71
5.9	Comparing Simulink results with Effectuator results	72
5.10	PID tuning parameter values at ($T_s = 1$)	73
7.1	The entire process of the new system	79

List of Tables

3.1	Comparison between TCP /IP and UDP /IP	17
3.2	Communication variables declaration	27
3.3	Test cases of the Effectuator and Simulink Communication process	36
4.1	The Effectuator and the Simulink model input flag cases . . .	40
5.1	Segment controller variable declarations	58
5.2	Control loop variables declaration	65

List of Abbreviations

ISW	Institute for Control Engineering of Machine Tools and Manufacturing Units.
DEVEKOS	Integrated engineering for secure, distributed and communicating multi-component systems.
FPGA	Field programmable gate array.
CPU	Central processing unit.
UDP	User Datagram Protocol.
TCP	Transmission Control Protocol.
DHCP	Dynamic Host Configuration Protocol.
CNC	Computer numerical control.
PC	Personal computer.
RTOS	Real-time operating system.
DLL	Dynamic-link library.
RAM	Random-access memory.
lwIP	lightweight IP.
PCB	Printed circuit board.
JTAG	Joint Test Action Group.
afag	Afag Automation AG (Machine industry company).
UART	Universal asynchronous receiver-transmitter.
USB	Universal Serial Bus.
pcb	Protocol control blocks.

Chapter 1

Introduction

1.1 Motivation

The world lives, currently in the era of Industry 4.0, in which there is a considerable change in the way by which we create products due to the extensive evolution in the digital systems. During the first industrial revolution era, the manufacturing process had to depend on the water and steam power. Then the electric power took the manufacturing process to a second era as it provided the possibility of creating the mass production and assembly lines during the second industrial revolution that was the start of the idea of the factors known today. Later on, the invention of computers made a breakthrough in the industrial systems represented in the automated production lines, so the manufacturing process became more efficient and productive. The extensive evolution in the network and communication systems provided the ability to integrate the cyber-physical systems with the Internet of systems and Internet of Things (**IoT**) that upgraded the third industrial stage to the fourth one that is known as Industry 4.0.

“For a current research project, cyber-physical, networked controllers are developed that offer ‘capabilities’ over the production network such that the execution details and internal protocols are encapsulated from the user [1].” The extensive development in the electronic devices in both of their considerable computing capacity and small sizes supported each of production components such as axes, end effectors, robots, and automated guided vehicles (**AGVs**) with its controller providing a kind of decentralization for system components control. That reduces the centralized control as “the

previous centralized control architecture places high time demands on the implementation of the individual steps [2].”

This decentralization process gives the manufacturing systems some advantages [3]:

- An easy usage of the component skills via a simple user interface.
- Actuating the system directly via its internal protocols.
- Eliminating the problems that could appear for different hardware and protocols via communicating common Ethernet connections.
- Replacing the limited specific components with a planned system installation that has universal capabilities.
- Creating different levels of skill complexity with the combination of different components into subsystems.

The motivation of working on the **DEVEKOS** research project is that controllers for the decentral interpolation approach are not commercially available. For this purpose, a validation concept for distributed interpolation was developed using a MATLAB Simulink model, so these models can be converted into other hardware languages such as VHDL in further implementations [3].

1.2 Aim of thesis

This thesis aims to integrate a linear axis into a multi-agent system. A DB42S03 Brushless DC motor will actuate this axis in following works. One of the goals is to implement a suitable communication structure by which this axis can communicate with other components of the system so it can receive orders and offer its skills via this network of communication. The second goal is to find an algorithm that can use the received orders and make the axis respond to its environment. The main aim in this part is making the axis able to do the positioning task by dividing the path between its start and endpoint into several segments. The third goal is to build a position PID control algorithm that controls the motion execution of the motor along its way between the start and endpoint. A MATLAB/Simulink model will simulate this axis control algorithm. After that, it will be implemented using a C code that will run on an embedded NIOS board.

1.3 Requirements of thesis

The exposé, which was written before the thesis created the following requirements to complete the thesis:

- Using the **UDP** (**U**ser **D**atagram **P**rotocol) communication protocol proposed by the previous work and building a communication network between an embedded NIOS hardware board called the Effectuator and the Simulink model that simulates the original system [4].
- Integrating the Effectuator into the original system consisting of the other axes. This system will be illustrated in chapter 2.2.
- Creating and simulating a PID position control algorithm for a linear axis using MATLAB Simulink platform.
- Implementing the PID control algorithm using C code and running the code on the Effectuator.
- Converting the logic of the Simulink model of the original axes to a C algorithm running on the Effectuator.

Chapter 2

Analysis of previous work on DEVEKOS research project

2.1 Fundamental concepts

This chapter 2.1 will discuss some fundamental concepts related to previous work on the DEVEKOS research project. It will discuss the skill based-axes concept and how it can be integrated into a more complex system called Holon. It will also discuss the concept of decentralizing the production sub-systems by using distributed motion controllers instead of one central motion controller. It will also discuss the components path interpolating technique and the different challenges to achieve such a decentralized system.

2.1.1 Skill based-axes concept

The skill based-axes are axes in which controllers, software, sensors, actuators, and communication modules are embedded. Each axis encapsulates its internal issues but abstractly offers its skills via a communication network. A cross-coupled control concept was developed to achieve synchronization between more than two axes, for example, in tasks as pick-and-place tasks [5].

The skill-based-axis obtains knowledge about its environment in addition to its basic knowledge via the sensors. After that, it acts on the environment via its actuators to lead it into a sequence of states. A better sequence of states gives better performance of the axis represented by what is called the performance measure. Four parameters determine the task environment of

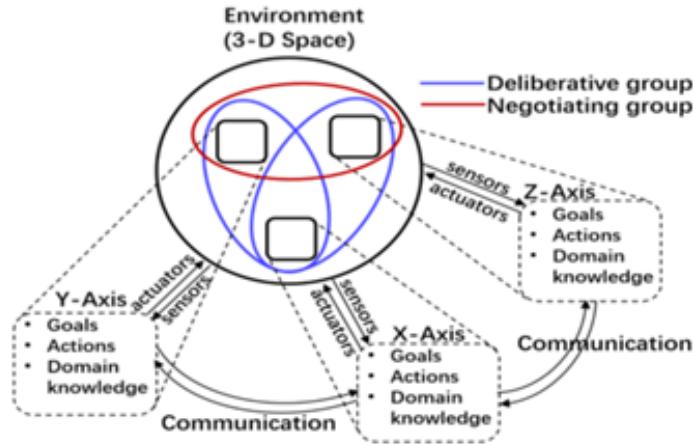


Figure 2.1: Cooperation types of a multi-axis system: negotiating and deliberating groups in a multi-agent system [5]

the axis described by Russell: PEAS (Performance measure, Environment, Actuators, Sensors) [5].

As shown in Figure 2.1, there are two types of cooperation between axes in multi-axis systems. The first one is a deliberating group in which two neighboring axes (X-axes and Y-axes) can communicate with each other, so they can reach a common goal. The second type is a negotiating group in which two axes (Y-axes and Z-axes) do not communicate directly. However, they send modifications to a third neighboring axis (X-axis).

Broadcasting messages to all axes is not a good choice for the communication part because there are non-communicating axes, for example, the two axes of a deliberating group [5]. This communication concept was one of the designed solutions. However, it was not selected for the current project implementation. In the current implementation explained by [6] and [7], communication between the axes is an address-based type. It could be (**TCP**) type or (**UDP**) type.

2.1.2 Holonic Manufacturing Control

Another important concept to be discussed is (**HMC**), which stands for Holonic Manufacturing Control. It means that a single Holon can consist of a group of components [8]. Based on this approach, each component has a

certain communication level by which it can offer its skills. The component level classifies the offered skills into low-level and high-level skills where the high-level skills are complex skills consisting of a sequence of low-level skills. This kind of multi-agent system helps components of different manufacturers to work and communicate with each other.

2.1.3 Decentralization concept

The production components or subsystems should be decentralized so that they have more responsibilities or tasks to do. One central computer that holds all the components data is to be replaced by control algorithms with different miniaturized electronic devices that are responsible for giving the components the right actuation at the right time. That gives a decentralized control for the system components where a high amount of data should be transferred between them to achieve cooperation with each other in doing parallel tasks. Transferring that amount of data increases the communication load for such systems and decreases the performance of the tasks, so a minimum number of state variables should be defined to represent the information of components status and tasks [6].

2.1.4 Distributed motion controller concept

There are control algorithms used to achieve motion control of agents in a synchronized way. For example, master-slave control is a control algorithm that depends on building a hierarchical structure in which agents or axes are not considered to be equal. However, it is easier to deal with all agents as they are the same. That makes their reconfiguration simple and eliminates the use of master-slave functionality [6]. These algorithms should meet the requirements of pick-and-place applications and be compatible with the commercial actuators drives, contour accuracy, and dynamics of the required tasks. The stated algorithms should also solve the trade-off that could happen between the computational and communication load. At the same time, these algorithms should be simple, so their implementation on real hardware and controllers is applicable [6].

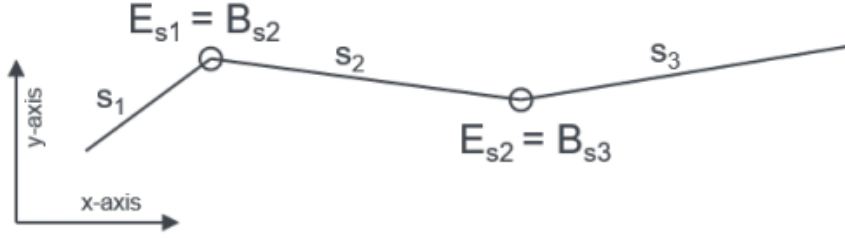


Figure 2.2: Notation for set point segments S_m [6]

2.1.5 Interpolation concept

A kinodynamic problem is dividing a path of multiple dimensions into a path of one dimension for each participant axis and determining the linear motion of the axes based on its acceleration, deceleration, and velocity. This kinodynamic problem should be solved to achieve tasks as interpolating a desired path so that pick and place targets could be accomplished [6]. Doing this task needs solving an optimization problem, where the axes collect their local data into one global data set, so they can find an optimized solution for their times and velocities to execute each segment of the interpolated path. Solving this kinodynamic problem is done centrally in the conventional **CNC** machines. However, in a decentralization approach, every axis equipped with its controller should provide its data to a common communication channel where the derivation of a coordination strategy and decentralized control structure can solve the problem [6]. The interpolation operation is done so that each axis has suitable speed compared to other axes along the desired path. As shown in Figure 2.2, a sequence of k points defines the desired interpolation path, these points are connected via $k-1$ linear segments S_m with a start and end point for each segment. Then the axes velocities are defined with the equation shown in equation 2.1 [6]:

$$v_{gen, s_m} = \frac{dp_{i, s_m}}{dt} = \frac{dp_{j, s_m}}{dt} \quad \forall i \neq j \quad (2.1)$$

Where a progress variable $P, s_m \in [0, 1]$ and the path function $f(P, s_m = 0) = B, s_m$ and $f(P, s_m = 1) = E, s_m = B, s_{m+1}$. Taking into consideration that both axes reaches the desired position at the same time following equation 2.2 [6]:

$$t(P_i, s_m = 1) = t(P_j, s_m = 1) \quad \forall i \neq j \quad (2.2)$$

2.1.6 Implementation challenges

By analyzing the current implementation, the following four main topics should be handled during further implementations. These topics include communication, time and synchronization, requirements on the control task as well as scaling and capacity [9]. The communication architecture should be chosen so that the agents can cooperate, negotiate, and solve conflicts between each other. Time delays, corrupt, missing messages, and interrupted communication amongst the components should be handled by using a safe action strategy [9].

A good synchronization architecture should also be chosen so that the components can perform parallel tasks. Control tasks determine the way the component can act to perform a requested skill. For example, if an interpolation task is requested from an axis, it should be able to compute how and where its transformation could be done. It should also be able to store and share information such as its position, dimensions, and direction of movement. At the same time, the real-time application problems such as collisions with or distances to nearby components should be computed synchronously [9].

Finally, the computation capacity of the electronic unit should be taken into consideration by developing new multi-agent-systems concepts that can handle thousands of agents during the running of the industrial application [9].

2.2 Initial setup

This chapter will discuss the initial setup of the thesis project, including a three-dimensional system called the demonstrator, the Simulink model used to simulate the demonstrator, and the embedded board called the Effectuator that will be integrated into the demonstrator setup. Chapter 2.3 will also give an idea about the desired system implemented by the thesis.

2.2.1 The demonstrator

A hardware setup called the demonstrator existing at **ISW**. As shown in figure 2.3, it consists of four real axes and a simulation **PC**. The axes are X-axis, Y-axis, and Z-axis as transitional axes representing the three-dimensional Cartesian coordinate system and C-axes as rotational axes about Z-axis. The

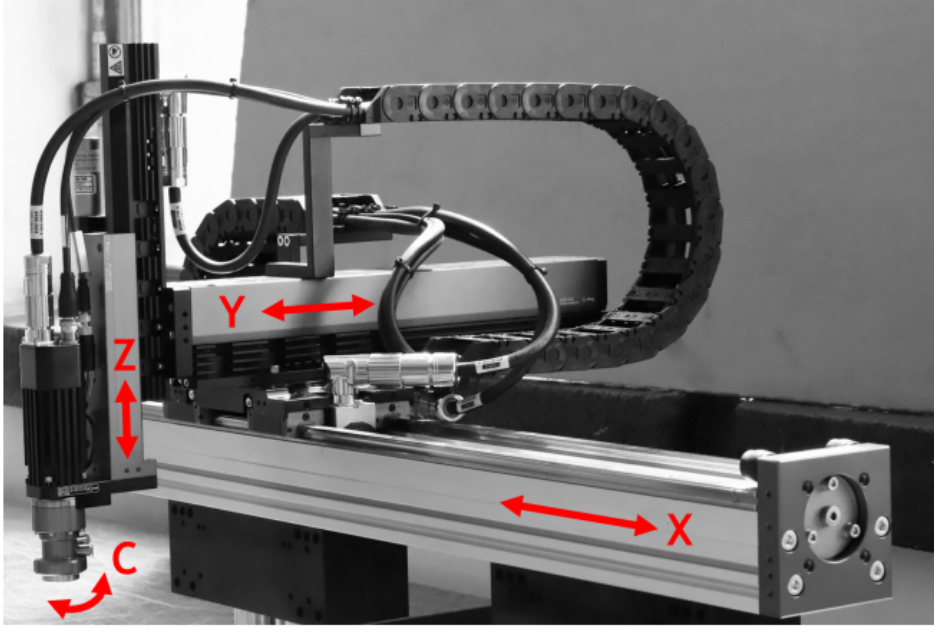


Figure 2.3: The demonstrator existing at **ISW** [6]

axes are driven by LinMot motor servo amplifiers that take the commands from the Simulation **PC** over a **UDP** interface.

2.2.2 The Simulink model

The Simulink model simulates the task done by the real axes, which is tracking a given curve in three-dimensional workspace, as shown in figure 2.4.

The Simulink model consists of the following Simulink blocks:

- **Agents/State flow:** This State flow block contains three state flow charts representing the entire process for the three main axes Axis-X, Axis-Y, and Axis-Z. It also contains the flowchart of the **dispatcher** that controls the flow of the entire process. It also contains some MATLAB functions used by the three axes to do the path interpolation, generate the **coordination vector** and the **consensus vector**, which will be illustrated as the following:
 - **Dispatcher:** It is a simulating agent that handles the communication network between other agents and controls the agents'

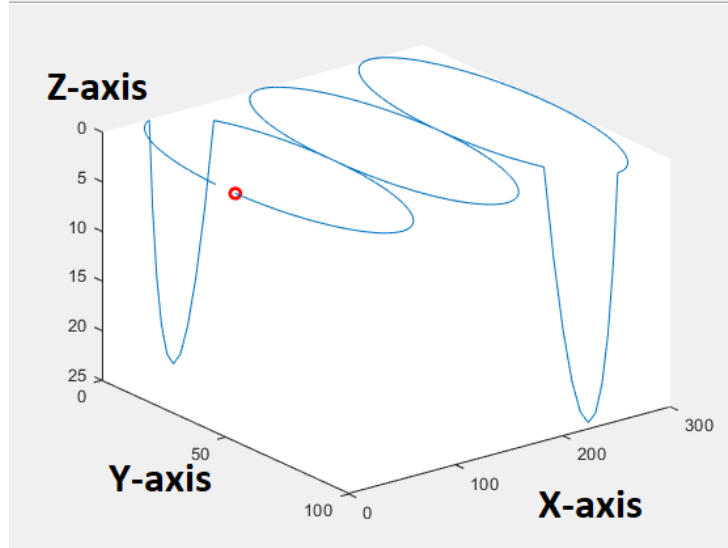


Figure 2.4: The input curve tracked by the transitional axes

transition between different states in their process. In the final work, it will be replaced by a communication network between all agents.

- **Coordination vector:** It is a vector calculated and provided by each axis, where the axis calculates the time it will take to execute each segment in its paths.
- **Coordination matrix:** It is a matrix where the dispatcher stores coordination vectors of all axes and shares it with them.
- **Consensus vector:** It is a vector stores the maximum time of each segment. This maximum time corresponds to the slowest axis in each particular segment.
- **Agents/Controlled systems:** It consists of three Simulink blocks for the three transitional axes. Each block contains the motion control loop for the axis.
- **Joystick Ctrl:** It contains a Joystick **GUI** (Graphical user interface) to control the speed of each axis.
- **Output-Logging and Animation:** It contains blocks to visualize the motion execution.

2.2.3 The Effectuator

The previous work downloaded a hardware design to the Effectuator to configure it with the Nios II hardware system. This step was implemented using the **Quartus** Software tool. Then a C project was created using **Nios II** Software Build Tools for Eclipse. The eclipse workspace was supported with the following:

- **UDP** communication libraries.
- **LED** (Light emitting diode) libraries to manipulate the Effectuator on-board **LEDs**.
- Free **RTOS** libraries to support real-time task execution.
- **UDP** send and **UDP** receive tasks. Each of them contains basic **UDP** communication commands.
- **Task_Agent()** function that controls the entire process of the Effectuator. It had a rough implementation that did not interact with all states of the Effectuator process. However, the current work completed the implementation of that function, so it can go through the entire process with the help of some individual functions that will be discussed later in chapter 4.3.2.

2.3 The system implemented by the thesis

Figure 7.1 shows the desired system implemented and explained by the upcoming chapters of the thesis. The Effectuator and the demonstrator simulated by the Simulink model have a **UDP** communication structure between them, so they can communicate their coordination data with each other before and within their motion execution. The Effectuator drives a new linear axis called X2-axis that lies on the X direction and does the same tasks done by the original X-axis but within a new predefined workspace. The Simulink model represents the original axes of the demonstrator. The process is going as the following:

1. **Register:** Each axis should register to the dispatcher, so the dispatcher knows the number of the axes in the system and the size of the coordination matrix.

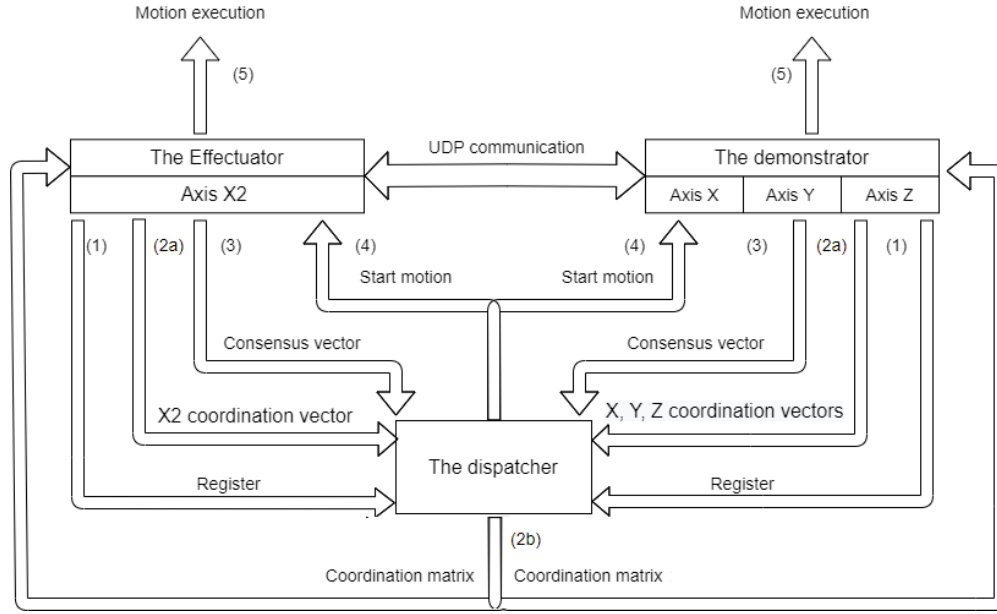


Figure 2.5: The entire process of the new system

2. (a) **Generate coordination vector:** Each axis receives the target curve path points and create its curve based on its defined workspace. Then it calculates its coordination vector and sends it to the dispatcher.
- (b) **Communicate coordination matrix:** The dispatcher collects the coordination vectors of all axes and arranges them in the coordination matrix. Then it share the coordination matrix back to all axes
3. **Generate consensus vector:** Each axis calculates its consensus vector based on the received coordination matrix and its generated curve and sends back to the dispatcher.
4. **Communicate consensus vector:** The dispatcher collects the consensus vector of all axes and allows them to start their motion execution.
5. **Start motion execution:** Each axis start the motion execution and communicate its position and segment index with other axes during the motion execution.

Chapter 3

Communication between Effectuator and the original Simulink model

This chapter will explain the implemented **UDP** communication, so the Effectuator and the demonstrator can share the coordination data explained in chapter 2.2.2. As shown in figure 3.1, the Effectuator implementation was divided into three main functions (**UDP_initiate**, **UDP_send**, and **UDP_receive**). The functions detailed explanation will be covered in chapter 3.8.1. A corresponding **UDP** communication block was created in Simulink. It consists of **UDP_send** block, and **UDP_receive** block that will be explained in chapter 3.8.2, and a communication stack that will be explained in chapter 4.2. The communication implementation allowed the Effectuator to be integrated into the other axes' working area via a function called **Task_agent()** that will be explained in chapter 4.3.2. This function acted as the link between the communication process explained in this chapter 3 and the motion control process that will be explained in chapter 5.

3.1 Industrial communication background

In the industrial field, most of the industrial systems have changed to be automated. The communication between automated devices in these systems became a critical concern to provide the control and the management of the industrial process.

3.2. Analysis of previous work on the communication task

A set of communication protocols that help in achieving communication between different devices or agents. These communication protocols are responsible for defining the rules that are responsible for data and information transfer and exchange between the agents. Various types of communication protocols differ in how they function (e.g., data flow, data routing, connection control, multiplexing, ordered delivery, flow control, error control). Some manufacturers tend to create their communication protocols that serve their specific devices and operations. In this case, they are called proprietary protocols. On the other hand, there are standard communication protocols that are created by the cooperation between experts of different organizations to serve multiple devices.

To be more specific industrial protocols are considered to be real-time communication protocols designed to interconnect the systems, interfaces, and instruments of which an industrial control system consists. Developers created many of them to communicate in a serial way over physical connections at low speeds. After that, they were improved to work over Ethernet networks using routable protocols (e.g., **TCP/IP**, **UDP/IP**) [10] between which there will be a detailed comparison in chapter 3.3.

3.2 Analysis of previous work on the communication task

This chapter 3.2 will analyze the communication process of previous work for both the Simulink model and the Effectuator done by [7] and [4] respectively.

3.2.1 The original Simulink model

The original Simulink model simulates the motion of the three transitional axes mentioned in the initial setup in chapter 2.2. For each of these axes, there is a LinMot **RTOS** drive that carries the motion commands from the Simulink model to the servo amplifier via an Ethernet communication structure that uses a **UDP** interface to drive the real axes of the demonstrator and the dispatcher that simulates the communication between the axes as it was mentioned in chapter 2.2.2.

3.2.2 The Effectuator

Communication interface

The previous work on this project has defined the communication interface the Effectuator should use to communicate with other agents. This communication interface has the following requirements [4]:

- Direct communication between agents.
- Non-continuous data transfer to save the Effectuator resources.
- The ability to integrate the Effectuator into the current demonstrator setup.
- The ability to integrate further agents into the setup.
- The compatibility of the chosen network protocol with the hardware and software features of the Effectuator.
- The compatibility of the chosen data type with the required accuracy of data transmission.
- The scalability of the message if the number of axes or path points is increased.
- The uniformity of the message type, so one port is used for one message data type.

Message structure

The previous work designed the message structure to fit and handle the different data transmitted between the Effectuator and the original model, for example, path points, coordination, and consensus vectors. It consists of three parts as the following [4]:

- **Message:** It is a single integer indicating the data content if it is path points, coordination vector, or consensus vector.
- **Data format:** It consists of two integers indicating the data dimensions in terms of rows and columns, respectively.

3.3. Comparison between **TCP** and **UDP**

- **Data:** This part is the message content. It consists of several integers, which are the columns of the matrix in a consecutive sequence.

Matrices are transmitted column by column linearly concatenated beside each other to fit in the one-dimensional array used by the Effectuator in data transmission. The previous work chose the 32-bit integers, and **UDP/IP** communication protocols for the implementation for Effectuator resource-saving purpose and the following two sections 3.3 and 3.4 will explain the theory behind choosing **UDP/IP** as communication protocol instead of **TCP/IP** for this implementation.

3.3 Comparison between **TCP** and **UDP**

The following table 3.1 will construct a detailed comparison between **TCP/IP** and **UDP/IP** communication protocols to figure out which of these two protocols is more suitable for the communication task implemented on the Effectuator [11].

The green color indicates that this protocol is better in this specific feature, unlike the red color.

Table 3.1: Comparison between **TCP/IP** and **UDP/IP**

Feature	TCP/IP	UDP/IP
Definition	Transmission Control Protocol (TCP) is a connection-oriented protocol by which devices can communicate across an internet network.	User Datagram Protocol (UDP) is a connectionless protocol that is similar to (TCP). However, it ignores error-checking and recovery services.
Data transfer	TCP establishes a strong connection between TCP server and client, which guarantees that the exchanged data are delivered to the client.	UDP is connectionless, which means it doesn't detect if the data reached the client or not.

Reliability	TCP is more reliable as acknowledgment messages are sent and received between the server and the client when they start and end the connection. It also provides retransmission of data if data loss occurs.	UDP doesn't provide acknowledgment messages or lost data retransmission so data loss can happen in transit.
Sequence	TCP delivers data to the client in the same order they were sent. If the order has changed during transmission, TCP reorders data before they are delivered.	UDP doesn't guarantee that the delivery of data is in the same sequence of their sending.
Connection	TCP has heavyweight connections to deal with network congestion and packets size optimization, which reduces its performance.	UDP has a lightweight connection as error handling and order of messages are not required.
Speed	TCP is slower.	UDP is faster.
Header Size	TCP's header file size is 20 bytes.	UDP's header file size is 8 bytes.

After having a comparison between **TCP** and **UDP** in terms of main features, the diagrams shown in figure 3.2 and figure 3.3 give a comparison of the program flow of **TCP** and **UDP** for both server and client modes.

Through the comparison, it was clear that **TCP** is a connection-oriented protocol. However, **UDP** is connectionless. This gives **connect()** method two different functionalities in both **TCP** and **UDP** case as shown in figure 3.2.

Calling **connect()** before **send()** is a must in **TCP**. It establishes the connection between server and client and guarantees that a three-way handshake has happened between them before sending packets over the network. Through this three-way handshake process server sends a synchronization packet to the client. Then the client replies with an acknowledgment packet

3.3. Comparison between **TCP** and **UDP**

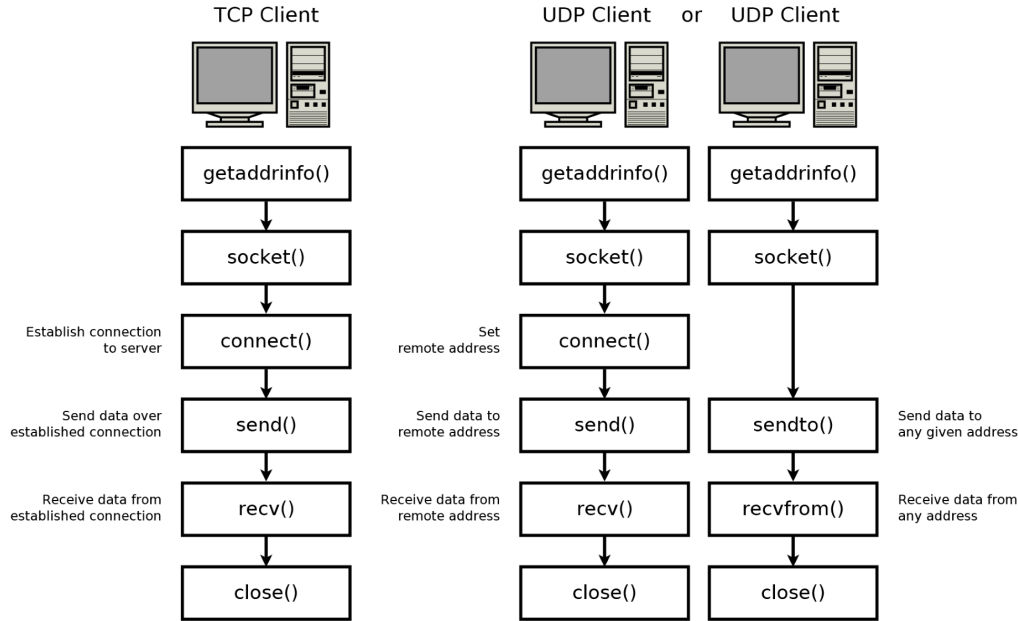


Figure 3.2: Comparison between **TCP** and **UDP** client program flow [12]

and a synchronization packet, then the server replies with an acknowledgment packet. This process ensures that both server and client are ready for the real data communication process.

Calling `connect()` works differently in **UDP**. It just informs the **UDP** client of a specific IP address and port number to which data will be sent using the `send()` method. This gives the **UDP** client the possibility of having another different structure where `connect()` can be ignored and replaced by `sendto()` or `recvfrom()` that include the IP address and port number to or from which **UDP** client can send or receive data [12].

Moving to the server mode, the **TCP** server program is different from the **UDP** server, as shown in figure 3.3. **TCP** server needs to call `listen()` and `accept()` methods to establish a new connection. However, the **UDP** server needs only to call the `bind()` method that assigns a local address to the connection process so it can immediately start the connection process. There is another difference between **TCP** and **UDP** server. **TCP** server needs to manage a socket for each client with which it communicates. However, the

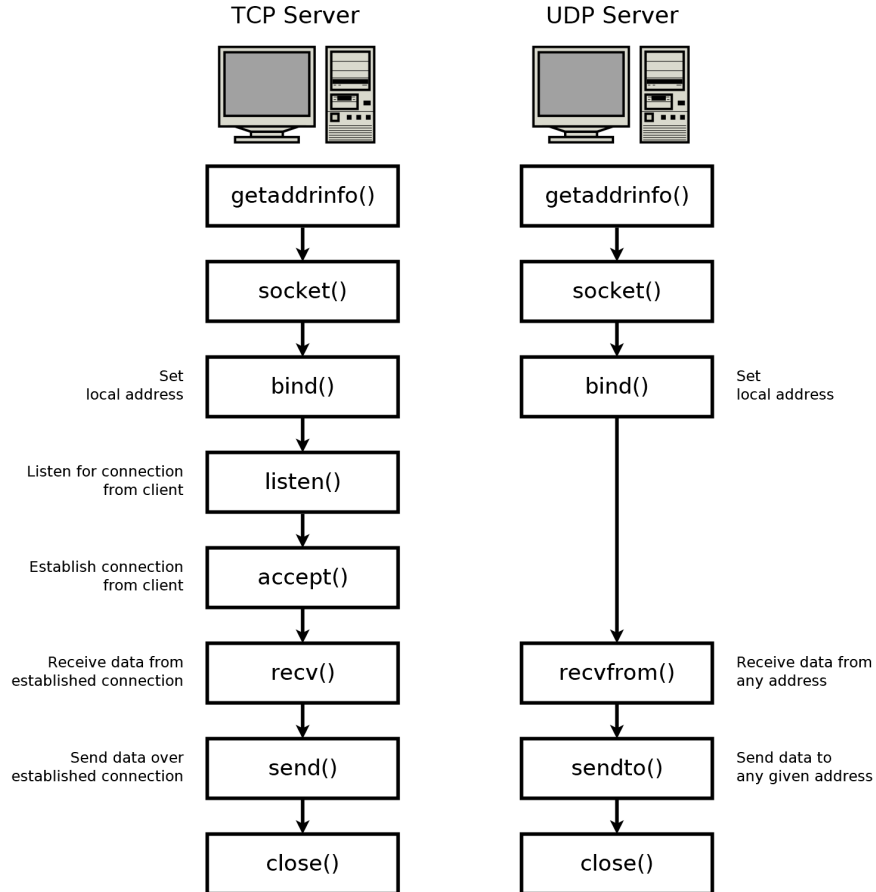


Figure 3.3: Comparison between **TCP** and **UDP** server program flow [12]

UDP server needs only one socket, and it can communicate with any number of clients. To be more clear the socket is an endpoint of communication between server and client through which all data are sent and received over the network.

3.4 TCP and UDP comparison conclusion

A conclusion from the previous comparison between **TCP** and **UDP** in both main features and program flow is that **TCP** is a standard communication

3.5. Lightweight IP stack (**lwiP**)

protocol that offers high reliability in the data flow, error handling and recovery for the lost data. That gives it an opportunity in the applications that require a high level of reliability where timing is not important. On the other hand, **UDP** is faster with lightweight connections that reduce the device resources and save it. To relate this conclusion with the application implemented by the thesis based on the recommendation of the previous work on the research project, **UDP** will be the selected communication protocol as it is faster and helps in saving the Effectuator resources [4].

Chapter 3.5 will talk about **lwiP**, which is the selected **UDP** software library used in the implemented communication structure.

3.5 Lightweight IP stack (**lwiP**)

lwiP is an open-source implementation of the **TCP**/IP communication protocol that was developed by Adam Dunkels at the Computer and Networks Architectures (CNA) lab at the Swedish Institute of Computer Science (SICS) but now, developers from all over the world develop it. At first, the purpose of **lwiP** was to decrease the **RAM** usage of embedded systems without affecting **TCP** main features. That gave the embedded system the possibility of saving its resources to do its main tasks parallel to the communication task. Currently, it supports many communication protocols (e.g., **TCP**, **UDP**, **DHCP**). It also supports the IP Internet Protocol with its two versions IPv4 and IPv6 and multiple network interfaces [13].

The following three sections will talk about the implementation of the communication task on the Effectuator, including the hardware setup, the software setup, and the usage of **lwiP** stack in the coding part in C language.

3.6 Hardware setup

The following list will include all the hardware components used in the setup supported with figure 3.4 where components are numbered.

1. The Effectuator

It is an electronic embedded system that consists of three connected PCBs (Printed circuit board). A **CPU PCB**, a power/communication/peripheral **PCB**, and a motion **PCB**. The main component of its **CPU** is an Altera **FPGA** (Field-programmable gate array) MAX10 chip that

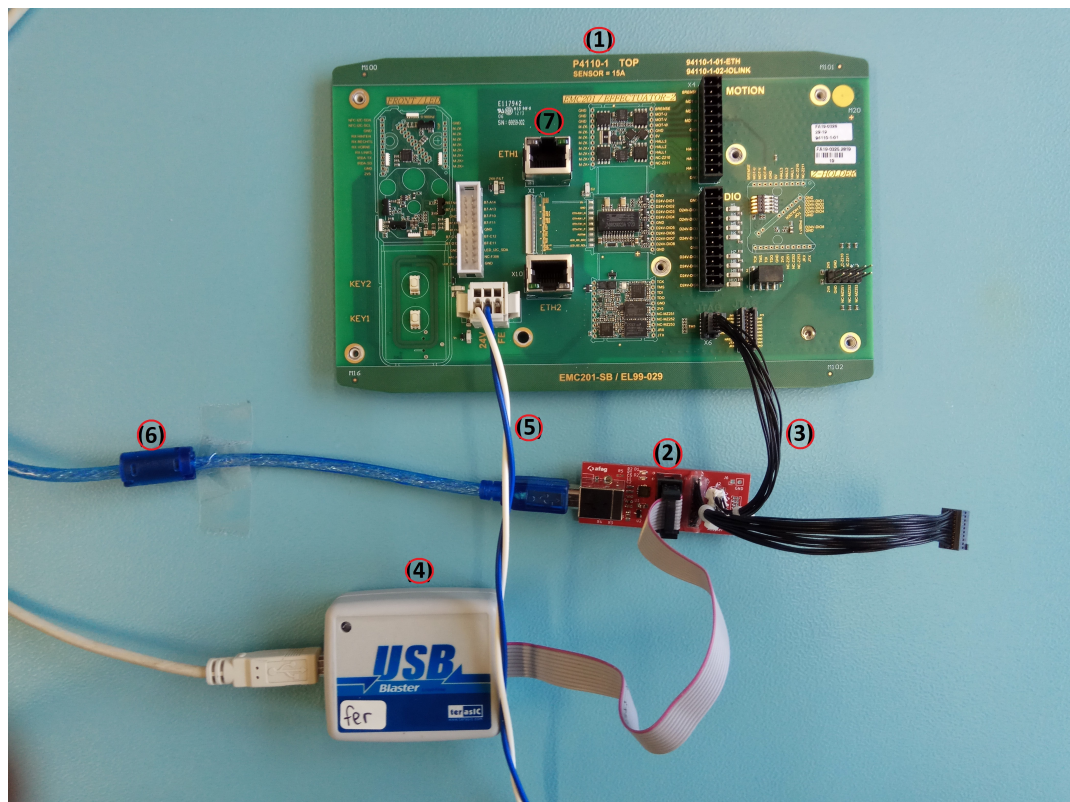


Figure 3.4: Hardware setup

3.6. Hardware setup

allows the integration of a complete system on one chip and decreases the required space on the **PCB**. On the second **PCB**, there are two installed Ethernet channels for communication purposes. There is also one **JTAG** (called after the Joint Test Action Group which created it) for programming and debugging. On the third **PCB**, there are input and output ports to which a real motor can be connected and driven by the motion control algorithm implemented in chapter 5

2. **JTAG** and **UART** Adapter

It is a red **PCB** produced by **afag** and used as an adapter for both the programming cable and the **UART** (Universal asynchronous receiver-transmitter) cable connected to the Effectuator.

3. **JTAG** cable

It is used to connect the **JTAG** adaptor with the Effectuator.

4. **USB** Blaster cable

It is a hardware unit used in embedded programming to download the code on the Effectuator. It is connected to the Effectuator by the **JTAG** and **UART** Adapter and connected to the simulation laptop by a **USB** (Universal Serial Bus) 2.0 Cable. It has a built-in blue **LED** (Light emitting diode), indicating that it is working.

5. DC power supply of 23V-29V and 10A

It is an AC/DC power supply used to supply the Effectuator. It supplies the Effectuator by 24 volts, which is regulated to 6 volts by the power **PCB** of the board.

6. **USB** 2.0 Cable

They are two serial **USB** cables. One is used to connect the programming cable to the simulation laptop, and one is used to connect the serial port on the **JTAG** and **UART** adapter to the laptop for debugging the code using **printf()** statements.

7. RJ45 cable

It is a type of cable used in Ethernet networking. It connects ETH1 port of the Effectuator with the simulation laptop as a physical communication tool between them.

The following chapter 3.7 will talk about the software setup used for the communication task implementation.

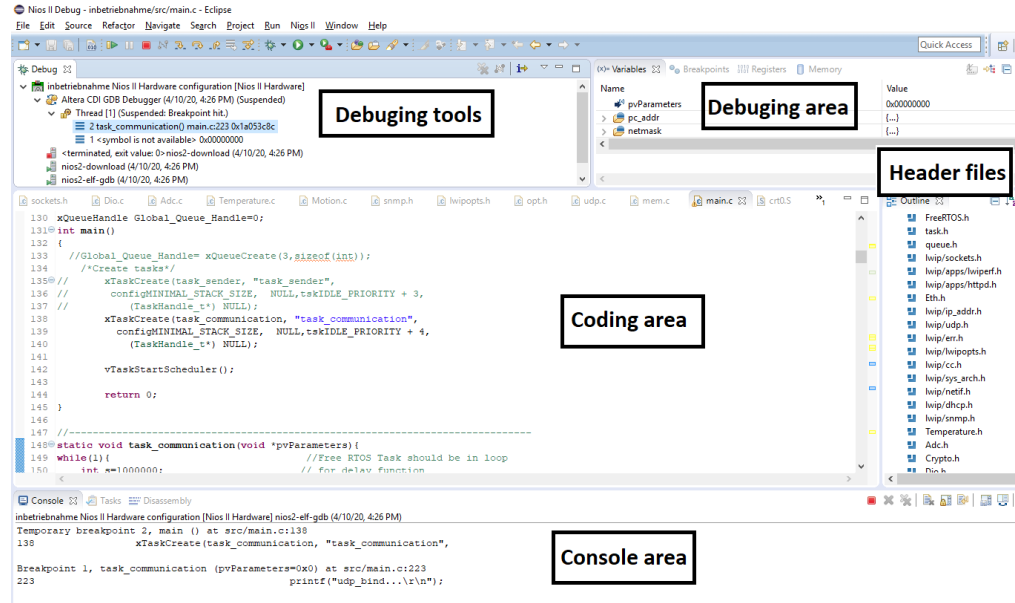


Figure 3.5: NIOS II debug perspective

3.7 Software setup

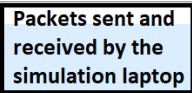
The following list will include all the software programs used supported by illustrating figures to their main features.

1. NIOS II Software Build Tools for Eclipse (Quartus Prime 18.1)

It is the tool chain where the code is developed and debugged. Its workspace includes the C files and header files that initialize the effec-tuator modules and **lwIP** files used in the communication task. One of its main features is the debug perspective that allows the hardware to go into the debugging mode and execute the code line by line based on the breakpoints put in the code, as shown in figure 3.5.

2. Wireshark Network Analyzer

It is an open-source packet analyzer used to see the **UDP** packets sent by the Effectuator to the laptop over the Ethernet cable with an indication of its source, destination, and length as shown in figure 3.6.



A specific packet details

Packet content



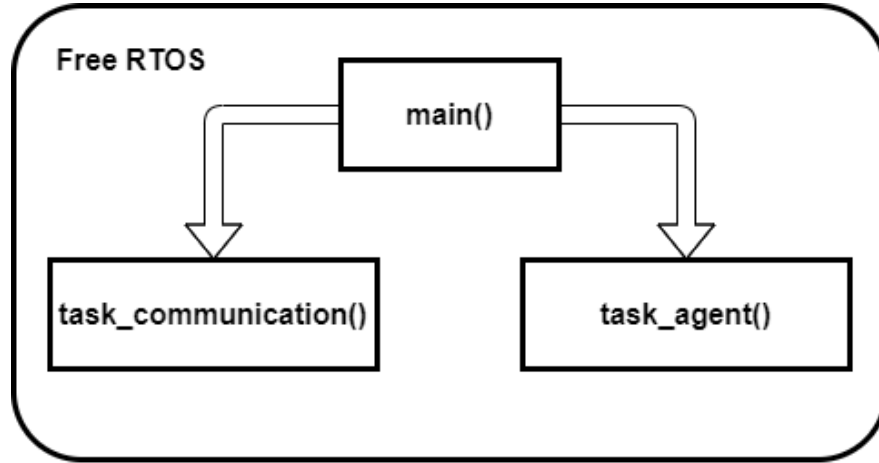


Figure 3.8: The Effectuator Free RTOS implementation

3. PuTTY Software

It is an open-source terminal. As shown in figure 3.7, used for debugging by displaying the output of the `printf()` statements within the code. That is done by connecting **USB 2.0** Cable between the laptop and the **UART** adaptor connected to the Effectuator. Then the laptop establishes a **UART** serial connection with the Effectuator via **COM4** port of the simulation laptop with a baud rate speed of 115200 bits/s.

3.8 Communication task implementation

This chapter 3.8 will explain the communication process implementation. The embedded implementation for the Effectuator and the Simulink implementation for the demonstrator.

3.8.1 Effectuator communication implementation

As shown in figure 3.8, the communication was implemented on the Effectuator as an **RTOS** task, which is the operating system that runs the code. It simply creates tasks and schedules them and runs a specific task based on its priority. The code was developed inside a method called `task_communication()`. This method was called by the `main()` method inside the **RTOS** structure. As shown in figure 3.9, the `task_communication()` calls other

3.8. Communication task implementation

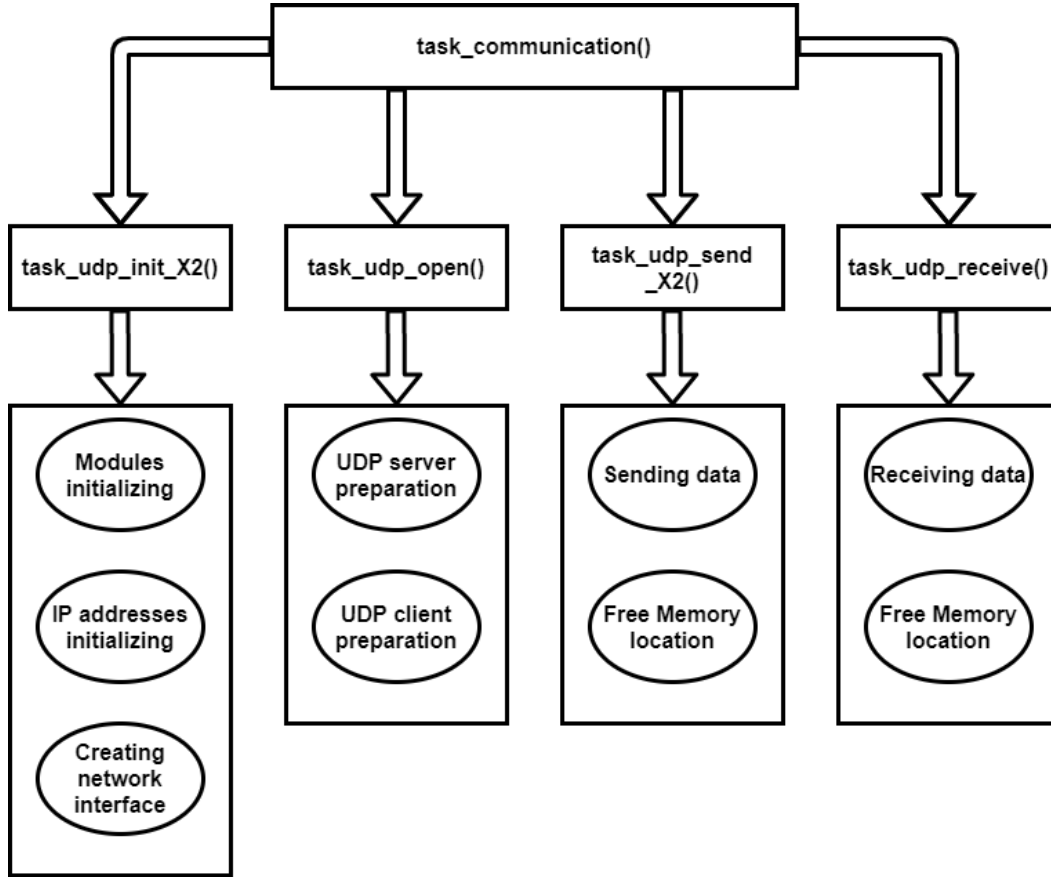


Figure 3.9: The Effectuator communication implementation

helper methods that execute sub-tasks needed to complete the communication process. However, before going with the explanation of the helper methods, table 3.2 declares the variables manipulated inside these methods.

Table 3.2: Communication variables declaration

Variable	Data type	Declared value	Description
<i>mac_address</i>	uint8_t array	{0x00, 0x30, 0x59, 0x12, 0x34, 0x56}	The Effectuator MAC address
<i>ip_settings</i>	tEthIpSettings	-	The Effectuator IP settings

<i>pc_addr</i>	ip_addr_t	192.168.5.90	The Simulation PC IP address
<i>eff_addr</i>	ip_addr_t	192.168.5.100	The Effectuator IP address
<i>netmak</i>	ip_addr_t	-	Network mask
<i>gw</i>	ip_addr_t	-	Network gateway
<i>netif</i>	struct netif	-	The network interface
<i>server_netif</i>	struct netif	-	The server network interface
<i>pc_port</i>	unsigned	8080	The PC port
<i>eff_port</i>	unsigned	8080	Effectuator sending port
<i>eff_port_recv</i>	unsigned	8080	Effectuator receiving port
<i>pcb</i>	struct udp_pcb	-	The Protocol control block carrying sent data
<i>statet</i>	u8_t	-	The return value of the memory allocation function
<i>p_send</i>	struct pubf	-	The sending pubf
<i>p_recv</i>	struct pubf	-	The receiving pubf
<i>lSocket</i>	int	-	It is a socket carrying received data
<i>inpflag</i>	int	0	The input flag
<i>bufferInput</i>	struct t_thing**	-	The input buffer
<i>outflag</i>	int	0	The output flag
<i>outBuffer</i>	struct t_thing**	-	The output buffer
<i>angle</i>	double	0	The sine waves argument
<i>first_iteration</i>	_Bool	true	First iteration flag

struct t_thing**

It is a custom struct created to carry the contents of dynamical arrays with the following data fields:

- **things**: It is a dynamical array holding the data of type **int**.
- **count**: It is an integer of type **int** that holds the length of the dynamical array.

It was created to store the length of a dynamical array once it is created instead of using the method **sizeof()** to get its length as it is not applicable for dynamical arrays in C language.

The rest of this chapter 3.8.1 will explain the different helper methods used to perform the communication process.

1. **task_communication()**

The implementation of this method abstract, where all the previous sub-tasks are called as following:

To start the communication process, both **task_udp_init_X2()** and **task_udp_open()** methods are called based on the value of a flag called *first_iteration*. If it is true, both methods are called and this flag is set false as it is not necessary to call these two methods again during code execution. Then, **task_communication()** calls **task_udp_send_X2()** and **task_udp_receive()** methods.

2. **task_udp_init_X2()**: This method is responsible for doing the following three sub-tasks:

(a) **Modules initializing**

This part of the code is responsible for initializing the different modules by calling the following methods:

- **stats_init()**: At first, this function is called to reset all statistics. These statistics are useful in debugging and performance tuning.
- **sys_init()**: It initializes the operating system emulation layer that maps the **lwiP** functions to the operating system-specific functions.
- **mem_init()**: It initializes all memory buffers.
- **memp_init()**: It initializes **lwiP** built-in pools.

- **udp_init()**: It initializes **UDP** protocol.
- **lwip_init()**: All the previous calls can be replaced by calling this method as it initializes all previous modules.
- **EthInit()**: It initializes the Ethernet port with the help of the Effectuator *mac_address* and its *ip_settings*

(b) **IP addresses initializing**

This part of the code initializes the IP addresses used in the communication process. Four IP addresses variables are declared that represent the addresses of the Effectuator, the simulation laptop, the network mask, and the default gateway with names (*pc_addr*, *eff_addr*, *netmask*, and *gw*). Calling **IP4_ADDR()** macro on these variables passes the specific values of IP addresses to them.

(c) **Creating network interface**

This part of the code creates a network interface. It is responsible for the interconnection between the Effectuator and the simulation laptop. Without this network interface, there was a problem that the **udp_sendto()** method returns an error message that it can't find a route to the destination address. The following paragraph will explain the way of creating this network interface .

At first, the method initializes a variable of type *netif*. Then there is a call of **netif_add()** method on it to add this network interface to the list of **lwIP** *netifs*. After that, this network interface is set as default network interface by calling **netif_set_default()** method on it. After that, this network interface becomes available by calling **netif_set_up()** method on it. Finally, the code has to wait for **netif_is_up()** method to return a true value, indicating that the network interface is ready to use.

3. **task_udp_open()**: This method is responsible for doing the following two subtasks:

(a) **UDP server preparation**

This part of the code performs preparations before the **UDP server** starts sending the data over the network interface created by **task_udp_init_X2()**. At first, it creates a new **UDP pcb** structure by calling **udp_new()** method and saves its return value into a variable of type *pcb*. For more illustration, *pcb* stands

for **Protocol control blocks**. It is used by the protocol layer to hold all pieces of information needed by the **UDP** or **TCP** socket (e.g., local IP address, remote IP address, local port number, remote port number, etc.) [12].

After creating the *pcb* structure, this part of the code relate the *pcb* structure with a local IP address and port by calling **udp_bind()** method on it. It is optional to bind the *pcb* structure to all available IP addresses in the Effectuator by passing the *IP_ADDR_ANY* in the **udp_bind()** method parameters. It was also important to bind the *pcb* structure to the network interface that was created by calling **udp_bind_netif()** method on it. Without this step, the same routing error mentioned in section (C) in **task_udp_init_X2()** will occur.

After binding the *pcb* structure with a local IP address, it will be connected to a remote IP address and port number by calling **udp_connect()** on it. As it was illustrated in chapter 3.2, calling **udp_connect()** method is optional in the **UDP** case as it is connectionless.

(b) **UDP client preparation**

This part of the code performs preparations before the **UDP client** starts receiving data. At first, it assigns the return value of **lwip_socket()** method to *lsocket* variable of type `int`. The mentioned method creates a socket and defines its type and the used protocol type. After that, it is related to the local IP address and port by calling **lwip_bind()** method on it.

4. **task_udp_send_X2()**: This method is responsible for doing only the following subtask:

(a) **Sending data**

After the preparation of the **UDP server**, the Effectuator is prepared to send data. This part of the code initializes a packet buffer represented by a variable *p_send* of type *pbuf* by calling **pbuf_alloc()** method. This packet buffer stores the data sent over the network. Then it assigns the data stored in the variable *outBuffer* to the *payload* field of *p_send*. Then it sends data from the Effectuator to the Simulink model over the network by calling the **udp_sendto()** method on it. Finally, *p_send* is freed by

calling `pubf_free()` method on it, so it can be used for sending further packets.

5. `task_udp_receive()`: This method is responsible for doing only the following subtask:

- (a) **Receiving data**

After the preparation of the UDP **client**, the Effectuator is prepared to receive data. This part of the code initializes a packet buffer `p_recv` the same way it was done in the sending part. Then it assigns a local variable `buffer` to the `payload` field of `p_recv`. Then it receives data from the Simulink model over the network by calling the `lwip_recv()` method on it. The received data carries the following three pieces of information:

- **Data length**: It will be stored in the `count` field of `bufferInput` variable.
- **Data content type**: It will be stored in the `inpFlag` variable.
- **Data content**: It will be stored in the `things` field of `bufferInput` variable.

Finally, `p_recv` is freed the same way it was done in the sending part.

3.8.2 Simulink communication implementation

As shown in figure 3.10, a block with called **UDP_communication** was created inside the original Simulink model to act as the other side of the communication process and communicating with the Effectuator as the start point of its integration into the original model as it will be explained in chapter 4. This block contains **UDP_send/receive** block for data sending and receiving and a **Communication stack** for incoming and out-coming data processing, which will be discussed later in chapter 4.2.

The **UDP_send/receive** block was created inside a **Function-Call Subsystem**, so each time of calling the **Communication stack** is called via a **Function-Call Generator**.

3.8. Communication task implementation

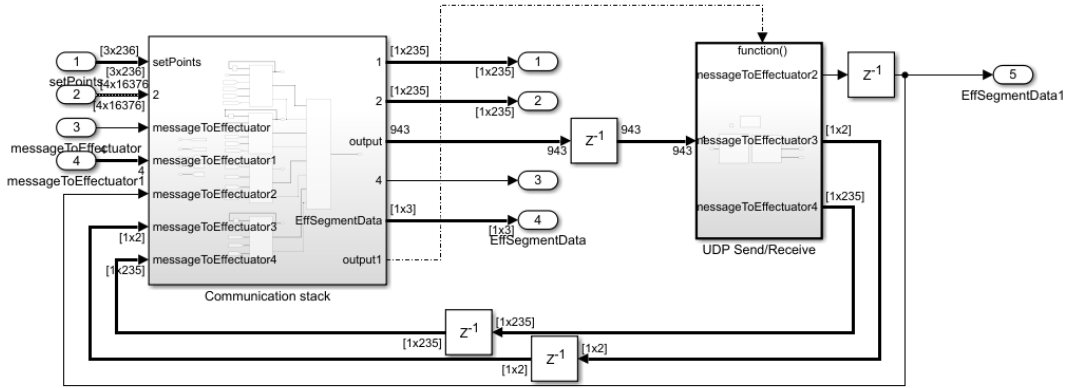


Figure 3.10: **UDP_communication** Simulink block

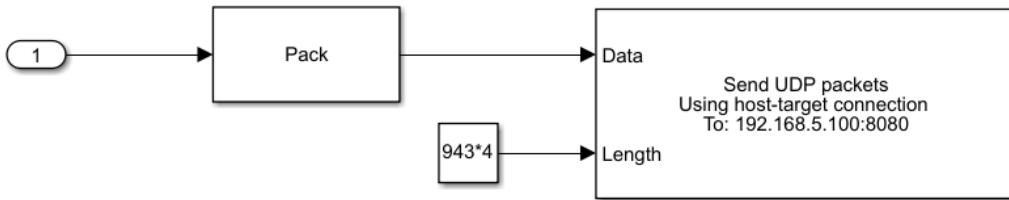


Figure 3.11: **UDP_send** Simulink block

UDP_send block

As shown in figure 3.11, this block contains a **Byte Packing** block that packs the int32 data into one single vector of uint8 and a **UDP Send** block that sends data to the Effectuator IP address (192.168.5.100) and port (8080).

UDP_receive block

As shown in figure 3.12, this block contains a **UDP receive** block that receives the data incoming from the Effectuator IP address, a **Byte Unpack** block that extracts the data and a MATLAB function that divides the in-

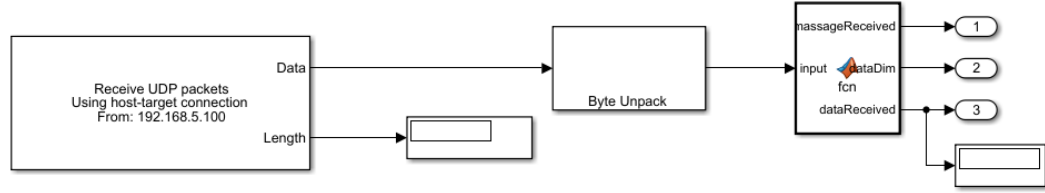


Figure 3.12: UDP_receive Simulink block

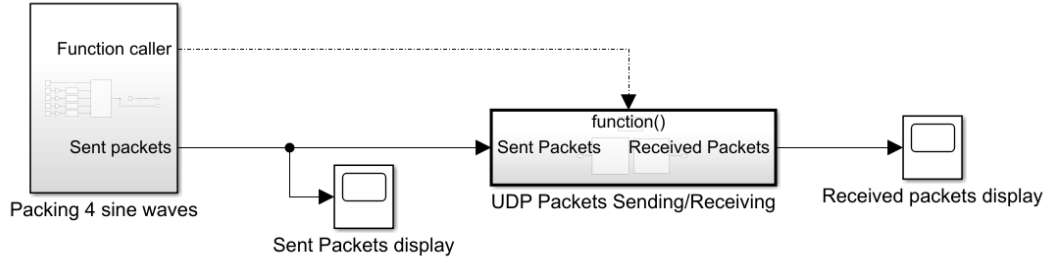


Figure 3.13: communicationTesting Simulink model

coming packet to the three parts of the message structure (Message, Data format, and Data) that was described in chapter 3.2.2.

3.9 Implementation test using sent and received sine waves

After the communication process between the Effectuator and the Simulink model was set up, the implementation was tested by sine waves transmitting between both sides. The reason to choose sine waves is that they are continuous repeated type of data so it can test the data transmission speed and smoothness. The Effectuator was to generate eight sine waves with phase shift $\pi/8$ between each and an increase of value 0.01 from 0 to 3.14 radians in each iteration using the function `sin()` supported in C. The sine waves were to be multiplied by 100 before sending it as an integer values instead of double values for resource saving as it was mentioned in chapter 3.2.2. On

3.9. Implementation test using sent and received sine waves

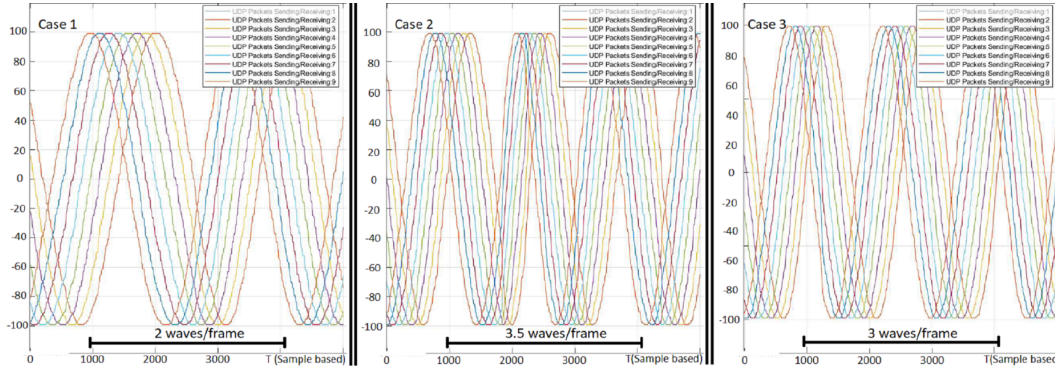


Figure 3.14: Number of received frames for test cases 1, 3, and 5

the other side a separate Simulink model called **communicationTesting** receives and displays them using a Simulink scope as shown in figure 3.13. It also sends four Simulink sine waves to the Effectuator that receives and displays their values using the PuTTY terminal.

3.9.1 Test results

The cases were divided into seven test cases with different sending/receiving combination between the Effectuator and the Simulink model as illustrated in table 3.3. The table also shows the Effectuator code elapsed time in seconds per each iteration. The eight sine waves were also displayed using a Simulink scope block and captured for the cases where the Effectuator is sending and the Simulink model is receiving at the same time as shown in figure 3.14. The figure shows also the number of received waves per one frame, where one frame represents 5000 of sample based time units.

3.9.2 Comments on results

The Effectuator

As shown in the test cases table 3.3, the test cases were divided into three groups as the following:

- **Case 1,2:** The Effectuator sends and receives messages to the Simulink model within an elapsed time 0.08275 seconds.

No.	Simulink	Effectuator	Code run time	Received waves /frame	PC Recv speed	PC Send speed
1	Send/Recv	Send/Recv	0.08275s	2	2.4kB/s	4.8kB/s
2	Send	Send/Recv	0.08275s	-	2.4kB/s	7.4kB/s
3	Send/Recv	Send	0.360612s	3.5	3.5kB/s	4.8kB/s
4	Send	Send	0.360612s	-	2.4kB/s	7.4kB/s
5	Recv	Send	0.360612s	3	3.5kB/s	-
7	Send/Recv	Recv	0.04746s	-	-	5.8kB/s
6	Send	Recv	0.04746s	-	-	7.4kB/s

Table 3.3: Test cases of the Effectuator and Simulink Communication process

- **Case 3,4,5:** The Effectuator only sends messages to the Simulink model within an elapsed time 0.360612 seconds.
- **Case 6,7:** The Effectuator only receives messages to the Simulink model within an elapsed time 0.04746 seconds.

These results follow the following inequality 3.1 representing the time elapsed for each group :

$$Receiving < Sending/Receiving < Sending \quad (3.1)$$

Only sending consecutive **UDP** packets takes longer time than sending and receiving packets.

The Simulink model

As shown in the test cases table 3.3, the test cases were divided into three groups as the following:

- **Case 1,3:** Simulink model is sending and receiving data with sending speed of 4.8kB/s.
- **Case 2,4,7:** Simulink model is only sending data with sending speed of 7.4kB/s.
- **Case 6:** Simulink model is sending and receiving data with sending speed of 5.8kB/s where the Effectuator is just receiving data.

3.9. Implementation test using sent and received sine waves

These results follow the following inequality 3.2 representing the sending speed for each group:

$$Sending/Receiving < Sending \quad (3.2)$$

Only sending consecutive **UDP** packets is faster than sending and receiving packets.

3.9.3 Communication testing analysis

By analyzing the Simulink cases, they can be divided into three groups where the Effectuator is functioning the same inside each group, so the Effectuator behaviour is the variable parameter between the three groups. The three groups are as the following:

- **Cases 1,2,3:** The Effectuator is sending and receiving.
- **Cases 4,5:** The Effectuator is only sending.
- **Cases 6,7:** The Effectuator is only receiving.

By looking at the **PC Send speed** column, the Simulink sending speed is almost constant for the three groups with a value between 4.8kB/s and 5.8 kB/s in the Simulink sending/receiving cases and a value of 7.4kB/s in the Simulink sending cases. However, by looking at the **PC Recv speed** column, the Simulink receiving speed is variable for the three groups based on the Effectuator behaviour with a value of 2.4kB/s in the Effectuator sending/receiving group and a value of 3.5kB/s in the Effectuator sending group. This means that the Simulink Sending speed does not depend on the Effectuator behaviour. However, the Simulink receiving speed is directly proportional with the Effectuator behaviour. The Simulink receiving speed is slower when the Effectuator is doing the sending and receiving tasks. This confirms the **UDP** feature that the server (Simulink) sends **UDP** packets and does not care if the client (Effectuator) receives them. However, when the Effectuator takes the (server) position, its behaviour affects the Simulink receiving speed (client).

3.10 Summary of the chapter

This chapter has covered the implementation of the communication process between the Effectuator and the Simulink model. It illustrated the communication usage in the industrial field. It analyzed the previous work on the communication process for both the Effectuator and Simulink. It made a comparison between the two different types of communication protocols (**TCP** and **UDP**) and concluded that comparison with choosing **UDP** for the current implementation. It illustrated the main tools for that implementation, which are the used software libraries, hardware setup, and software setup. It explained the current implementation and the implementation testing of the **UDP** communication for both the Effectuator and the Simulink model. Overall, the communication structure recommended by the previous work was implemented, so the Effectuator can now be integrated into the original system as it will be explained the following chapter 4.

Chapter 4

Effectuator integration with the original Simulink model

This chapter will discuss how the Effectuator was integrated and interacted with the original Simulink model. It will also discuss the tasks of the Effectuator, which was simulated with a separate Simulink model. The following section 4.1 will discuss the concept of the integration of new agents into the original demonstrator.

4.1 Analysis of previous work on new agents integration

The previous work has defined some requirements to integrate new agents. These requirements were as the following [4]:

- Superimposed axis directions: The new agent does not add a new dimension to the system. However, it superimposes one of the original axes within a defined workspace.
- Modularity: All agents are implemented in the same way, so creating and integrating new agents is easier.
- Scalability: Adding or removing new agents is easier.
- Interface: External agents and **PC** simulated agents can share their data.

InpFlag	Effectuator	Simulink
0	The dispatcher asks the Effectuator to register.	The Effectuator is ready to start the process.
1	The data received carries the path points.	The data received carries the Effectuator coordination vector.
2	The data received carries the coordination matrix.	The data received carries the Effectuator consensus vector.
3	The Effectuator can start motion execution.	The data received carries the Effectuator position and segment index.
4	The received data carries the Simulink axes position and segment index.	The Effectuator has completed its part of motion.

Table 4.1: The Effectuator and the Simulink model input flag cases

4.2 Incoming and out-coming UDP packets processing

As it was mentioned in chapter 3.8.2, **UDP_communication** Simulink block contains **Communication stack** handling the data sent and received between the Effectuator and the original Simulink model based on the incoming flag as clarified in table 4.1, where the Effectuator input flag is the Simulink output flag and vice versa. The **Communication stack** contains five MATLAB functions working as the following:

- **Get_coordination_vector**: As shown in figure 4.1, this function is doing two tasks. It generates an output variable carrying the path points of the input curve, the dimensions of the input curve, and a flag equals 1 according to the message structure, which was explained in chapter 3.2. This output variable will be sent to the Effectuator only if the incoming flag from the Effectuator equals 0 and the incoming flag from the dispatcher equals 0, which means the Effectuator is ready to receive the path points of the input curve and all axes have registered with the dispatcher. The second task of this function is to store the coordination vector sent by the Effectuator if the incoming flag from the Effectuator equals 1, which means that the Effectuator has calculated

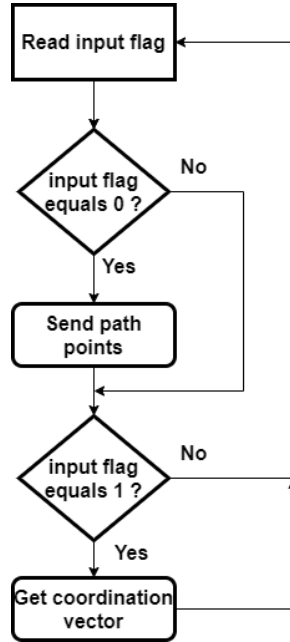


Figure 4.1: `Get_coordination_vector` function behaviour

and sent its coordination vector.

- **Get_consensus_vector:** As shown in figure 4.2, this function is doing two tasks in the same way as **Get_coordination_vector** does. It generates an output variable carrying the coordination matrix, the dimensions of the coordination matrix, and a flag equals 2. This output variable will be sent to the Effectuator only if the incoming flag from the Effectuator equals 1 and the incoming flag from the dispatcher equals 1. That means the Effectuator is ready to receive the coordination matrix, and the dispatcher has generated the coordination matrix after it has received the coordination vector from all axes. The second task of this function is to store the consensus vector sent by the Effectuator if the incoming flag from the Effectuator equals 2, which means that the Effectuator has calculated and sent its consensus vector.
- **Detect_motion_start/end:** As shown in figure 4.3, this function is doing two tasks in the same way as **Get_coordination_vector** and **Get_consensus_vector** are functioning. It generates an output variable carrying no data, but it carries a flag equals 3. This output variable

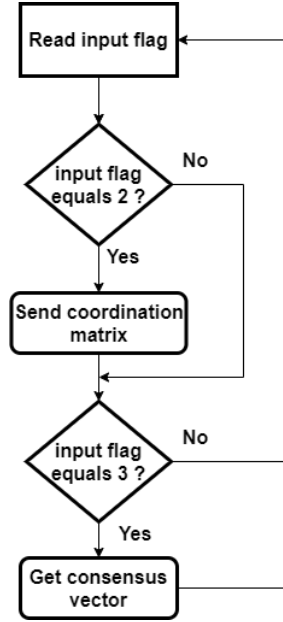


Figure 4.2: `Get_consensus_vector` function behaviour

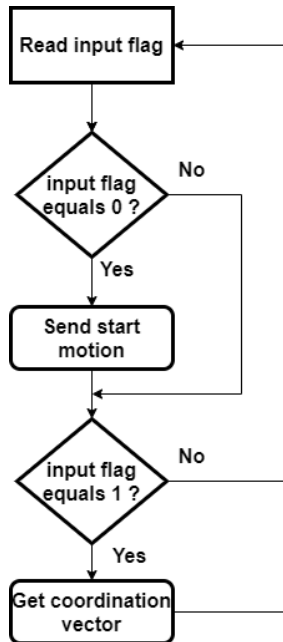


Figure 4.3: `Detect_motion_start/end` function behaviour

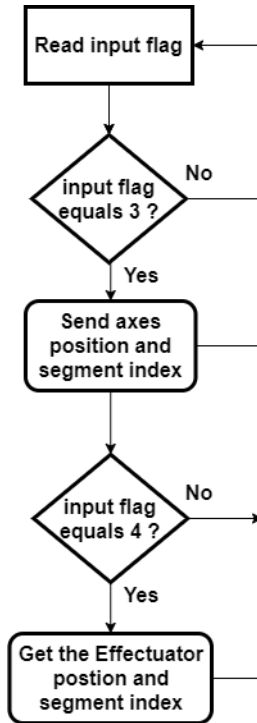


Figure 4.4: Send_axes_index/position function behaviour

will be sent to the Effectuator only if the incoming flag from the Effectuator equals 2 and the incoming flag from the dispatcher equals 2, which means the Effectuator and all other axes are ready to start the motion execution. The second task of this function is to check that the Effectuator has finished its work. That occurs when the incoming flag from the Effectuator equals 4. That means the Effectuator has completed its part of the motion.

- **Send_axes_index/position:** As shown in figure 4.4, this function is doing two tasks. It generates an output variable carrying the current position and segment indices of the three original axes, and the flag equals 4. This output variable will be sent to the Effectuator only if the incoming flag from the Effectuator equals 3, which means the Effectuator has executed its current segment. The second task of this function is to store that the Effectuator current position and segment index sent by the Effectuator.

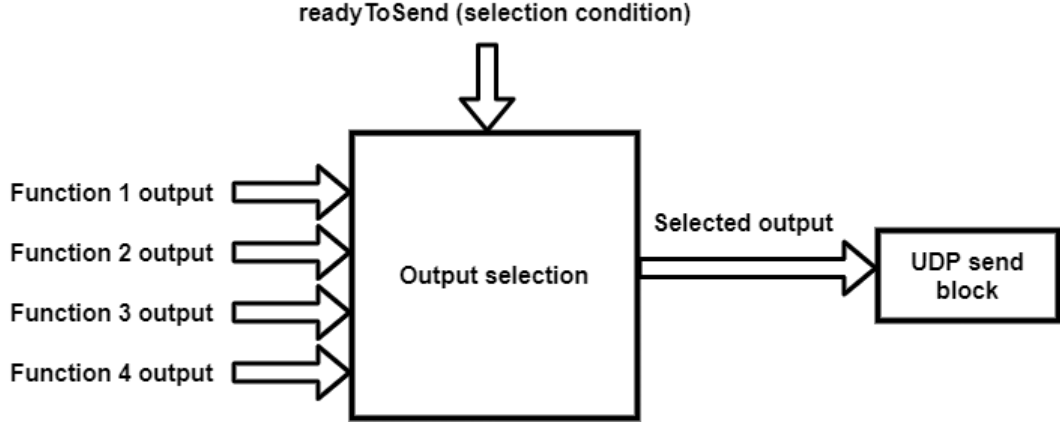


Figure 4.5: Output_selection function behaviour

- **Output_selection:** As shown in figure 4.5, this function is doing only one task. It selects only one output from the previous four MATLAB Functions to be passed as an input to **UDP_send** block. The selection is done based on a flag called *readyTosend*, which is handheld by each of the previous four MATLAB Functions to activate or deactivate its output variable.

UDP packets length

UDP Send Simulink block does not support changing the length of the vector to be sent during the model running. That means the length of the output vectors generated by the four MATLAB function has to be identical. The length of the longest expected vector was chosen to be the length of all output variables. It is the length of the coordination matrix sent by the Simulink model to the Effectuator, so this predefined length is calculated with the following simple equation 4.1

$$outputvariable_length = (N_p - 1)N_a + 3 \quad (4.1)$$

Where N_p is the number of path points for each dimension, N_a is the number of system axes, and the constant three is for the message structure (flag, number of rows and number of columns). For the current implementation $N_p = 236$, $N_a = 4$ so output variable-length = $(236 - 1) * 4 + 3 = 943$.

The same case is used for **UDP Receive** Simulink block. That means that the

4.2. Incoming and out-coming **UDP** packets processing

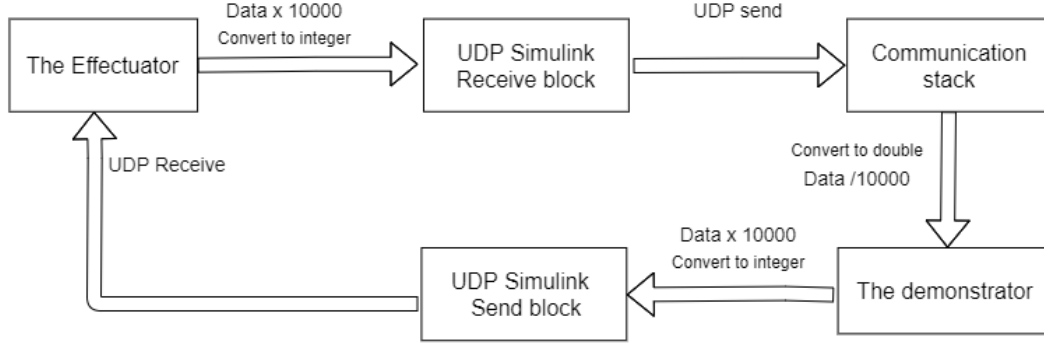


Figure 4.6: Data type conversion diagram

length of the input vectors received by the Simulink model should be unified with the longest expected length. It is the length of the coordination vector sent by the Effectuator to the Simulink model, so this predefined length is calculated with the following simple equation 4.2:

$$inputvariable_length = (N_p - 1) + 3 \quad (4.2)$$

where N_p is the number of path points for each Cartesian dimension, and the constant three is for the message structure (flag, number of rows, and number of columns). For the current implementation $N_p = 236$, so input variable length = $(236 - 1) + 3 = 238$.

Data type conversion

The data type of variables in the Simulink model is a double data type. However, it was recommended by the previous work to use **int32** data type during data transmission between the Effectuator and Simulink to allow sending more packets in one UDP message and save the Effectuator resources. For this reason, each message sent from Simulink to the Effectuator are converted to its thousands by multiplying it by 10000 and converting it to **int32** data type before the sending process. Oppositely, all messages sent from the Effectuator to Simulink are converted to **double** data type and divided by 10000 after the receiving process as shown in figure 4.6.

4.3 Effectuator insertion into the original common working area

This chapter 4.3 will explain the implementation used to integrate the Effectuator into the working area of the demonstrator axes. At first, it will explain the Simulink implementation added to the original Simulink model. Then it will explain the Effectuator implementation.

4.3.1 Simulink implementation

Agents/State-flow

As it was explained in chapter 2.2, the previous work has implemented a Simulink block called Agents/State-flow [7]. It is the common working area where the Effectuator should be inserted as a new agent participating in the process. The current work added a new flowchart, which is called Axis_X2 (the Effectuator). As shown in the figure 4.7, This flowchart is responsible for interacting with the **UDP** communication to let the Effectuator share its data to the common working area with the other axes. It is the same flowchart as the original axes except that it receives the coordination vector and consensus vector calculated by the Effectuator. It also does not do the motion execution. However, it waits until it receives a message indicating that the Effectuator has completed the motion execution to tell the dispatcher that the Effectuator is done.

PosTracer_v2

As it was illustrated in chapter 2.2.2, the previous work has created a Simulink block called PosTracer_v2 [7]. It updates the curve traced by the original axes, as it was shown in figure 2.4. As it is shown in 4.8, the current work added a MATLAB function called **DS_updateTraceEff** that updates the effectuator position received by **Send_axes_index/position** as it was explained in chapter 4.2.

4.3. Effectuator insertion into the original common working area

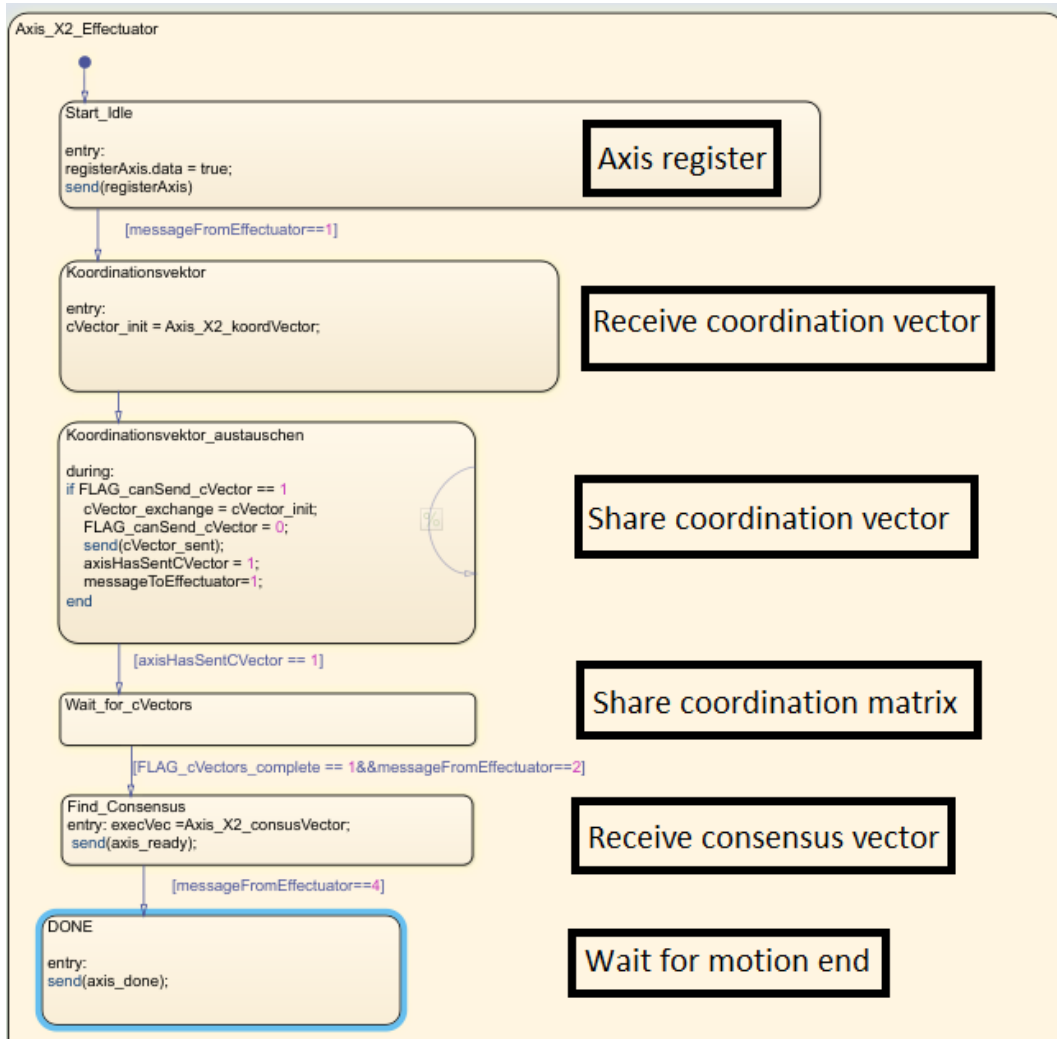


Figure 4.7: Axis_X2 _Effectuator flowchart

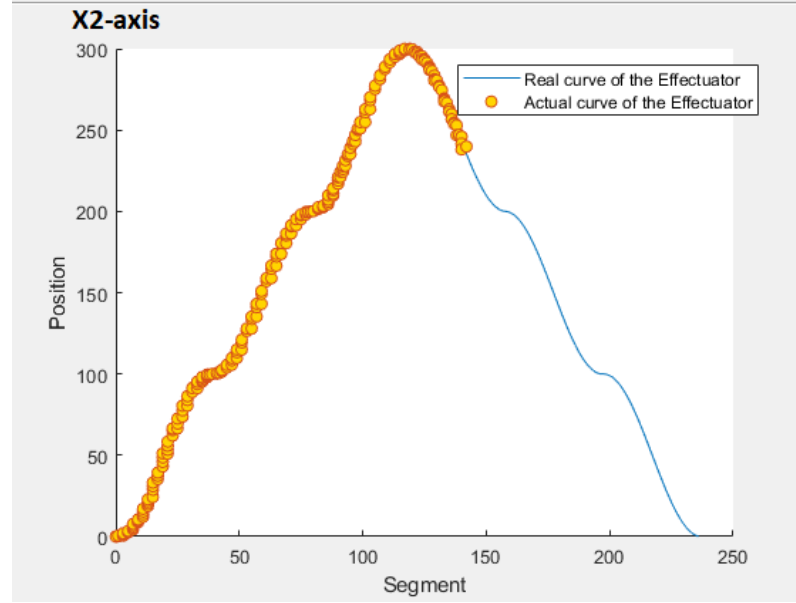


Figure 4.8: The curve traced by the Effectuator axis

4.3.2 Effectuator implementation

The previous work on the Effectuator has created the **task_agent()** function [4]. Its task is to call the functions, which are responsible for completing the Effectuator process starting from path planing until the motion execution is completed based on the input flag value. The current work calls **task_agent()** in the **main()** method inside the **RTOS** structure to be executed in parallel with the **task_communication()**. The following list will mention the functions implemented with the previous work then it will explain the new functions implemented by the current work.

- **generate_X2_curve()**: It generates the Effectuator curve after receiving the path points from the Simulink model. If the point is within the Effectuator workspace, it is a valid point. Otherwise, the Effectuator has to keep its current position.
- **generate_koord_vec()**: It generates the Effectuator coordination vector (explained in chapter 2.2.2) after getting its curve by **generate_X2_curve()** function.

4.3. Effectuator insertion into the original common working area

- **generate_consens_vec()**: It generates the Effectuator consensus vector (explained in chapter 2.2.2) after receiving the coordination matrix from the Simulink model.
- **generate_output()**: It generates the output to be sent by the Effectuator to the Simulink model. It does that by setting the output flag and the content of *outBuffer*.
- **generate_excution_vec()**: This function is implemented by the current work. It returns an execution vector of data type Struct **execVec**. It is a custom struct consisting of four dynamical arrays and their length as the following data fields:
 - **points**: It carries the Effectuator curve starting from the second point.
 - **speeds**: It carries the speed of each segment by getting the difference between the segment end and start point and dividing it by the corresponding segment time stored in the consensus vector.
 - **times**: It carries the time of each segment stored in the consensus vector.
 - **absolute_time**: It carries the absolute time of each segment by getting the sum of times for all previous segments.
 - **count**: It carries the length of each of the dynamical arrays. This length is identical for all arrays and equals the length of the consensus vector.

The functions also sets the variable *startPos* with the first point of the Effectuator curve

- **execute_movement()**: This function body is implemented by the current work. It creates a loop to execute the motion segment by segment. In each iteration, it updates the value of the current segment time, target_speed, and target_time for the new segment. Then it calls **go_to_pos()** method that is responsible for reaching the endpoint of the current segment. The implementation of **go_to_pos()** will be discussed later in chapter 5.4.5.

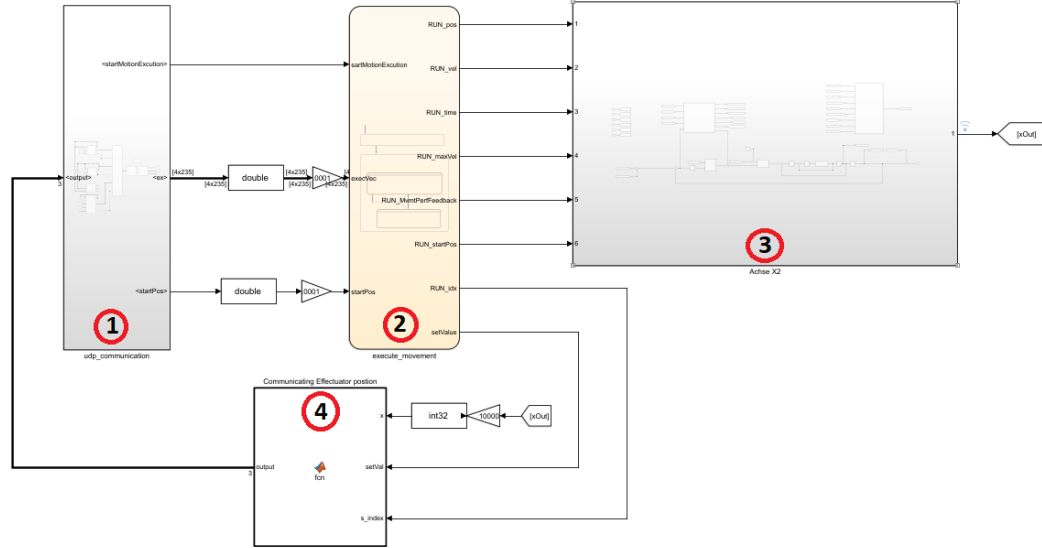


Figure 4.9: Effectuator Simulink model

4.4 Effectuator Simulation with a Simulink model

The current work created a separate Simulink model called **Effectuator**. It simulates all the tasks done by the Effectuator, including the **UDP** communication, the coordination and consensus vector calculations, and the motion control execution. As shown in figure 4.9, it contains three Simulink blocks, which will be explained in the following three subsections.

1. UDP_communication

It contains two **UDP** communication blocks one for sending and the other for receiving, which are implemented in the same way as **UDP_send** and **UDP_receive** blocks explained in chapter 3.8.2. It also contains five MATLAB functions to process the incoming and out-coming **UDP** packets as the MATLAB functions explained in chapter 4.2 but, this time, the data are processed from the Effectuator point of view as the following:

- **Generate_coordination_vector**: As shown in figure 4.10, this function is doing two tasks. At first, it receives the path points of

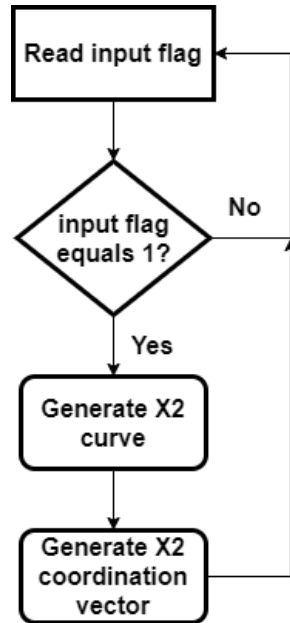


Figure 4.10: `Generate_coordination_vector` function behaviour

the Effectuator and generates the Effectuator target curve according to its predefined workspace dimensions. Then, it calculates the coordination vector of the Effectuator based on the generated target curve and outputs this coordination vector to be sent to the original Simulink model via the **UDP** communication network.

- **Generate_consensus_vector:** As shown in figure 4.11, this function is doing the following task. It receives the coordination matrix sent by the original Simulink model and generates the Effectuator consensus vector and outputs this consensus vector to be sent to the original Simulink model via the **UDP** communication network.
- **Generate_execution_vector function behaviour:** As shown in figure 4.12, this function is doing the following task. It receives the Effectuator target curve and consensus vector generated by the previous two functions and generates the Effectuator execution vector that holds the motion control data as it was explained in chapter 4.3.1 and outputs this execution vector to be used in the motion control part.

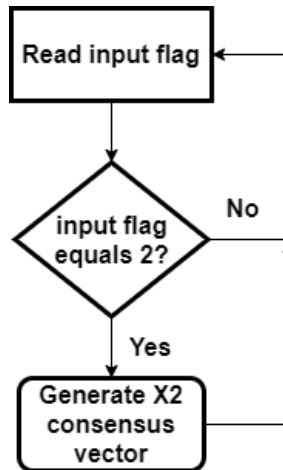


Figure 4.11: `Generate_consensus_vector` function behaviour

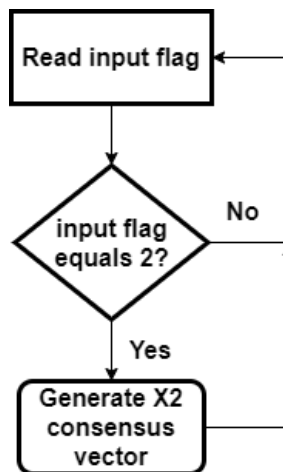


Figure 4.12: `Generate_execution_vector` function behaviour

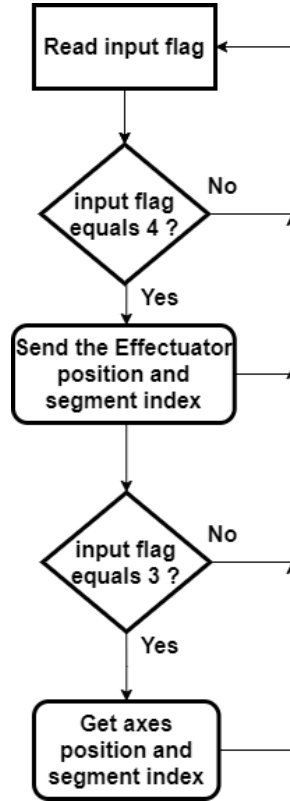


Figure 4.13: `Send.Effectuator_index/position` function behaviour

- **Send_Effectuator_index/position:** As shown in figure 4.13, this function is doing two tasks corresponding to the two tasks done by **Send_Effectuator_index/position** function explained in chapter 4.2. It generates an output variable carrying the current position and segment index of the Effectuator axis, and the flag equals 3. This output variable will be sent to the demonstrator only if the incoming flag from it equals 4. The second task of this function is to store that the axes current position and segment indices sent by the demonstrator.
- **Effectuator_Output_selection:** It is the same function as MATLAB **Output_selection** explained in chapter 4.2. It selects one output from the previous MATLAB functions to be sent via the **UDP** communication network.

2. **execute_movement**

It is a flow chart with a starting flag set by Effectuator MATLAB Function 3. It takes the generated execution vector and executes the Effectuator path segment by segment.

3. **Axis X2**

It contains the axis control loop that takes the motion parameter generated by the execute_movement and execute the motion for each segment. The following chapter 5 of motion control will explain the axis control loop in more detail.

4. **Communicating Effectuator position**

It is a MATLAB function that arrange X2-axis position and segment index into one linear vector to be sent and shared by the demonstrator axes, so they can be synchronized as it will be explained in chapter 5.5.

4.5 Summary of the chapter

This chapter covered the integration process of the Effectuator with the original Simulink model. It illustrated the requirements recommended by the previous work for the integration of the Effectuator and further agents. It explained the way of processing the UDP packets sent and received between the Effectuator and the original Simulink model over the communication network implemented in chapter 3. It also explained a Simulink model created to simulate the communication and motion control tasks done by the Effectuator. Overall, the integration process with the recommended requirements was implemented, so the Effectuator can perform the motion control execution that will be explained in the following chapter 5.

Chapter 5

Motion control for the new Effectuator axis

5.1 State of the art for motion control

One of the most important factors in the success of industrial systems is motion control. Industry 4.0 includes automated and digital systems that must achieve a high level of performance in doing different complex manufacturing tasks such as packing, material handling, and trajectory tracking. Reaching this high level of performance needs a powerful motion control to drive the motion actuators such as gears, motors, and motor drivers. These motion control programs should be simple and applicable to such systems to go in hand with the network and communication application to achieve the required targets [14].

5.1.1 PID controller

The idea of the PID controller is to decrease the difference between an output and a desired value. It keeps measuring this difference, which is termed as an error and adjusts its control signals to keep that error as small as possible. The simplicity of the PID controller made it one of the best control techniques used for many different industrial systems and plants. As shown in figure 5.1, a PID controller has three control parameters as the following [14]:

- **P or Proportional gain:** It simply multiplies the calculated error by a constant K_p to decrease it. An overshoot can happen in case of high

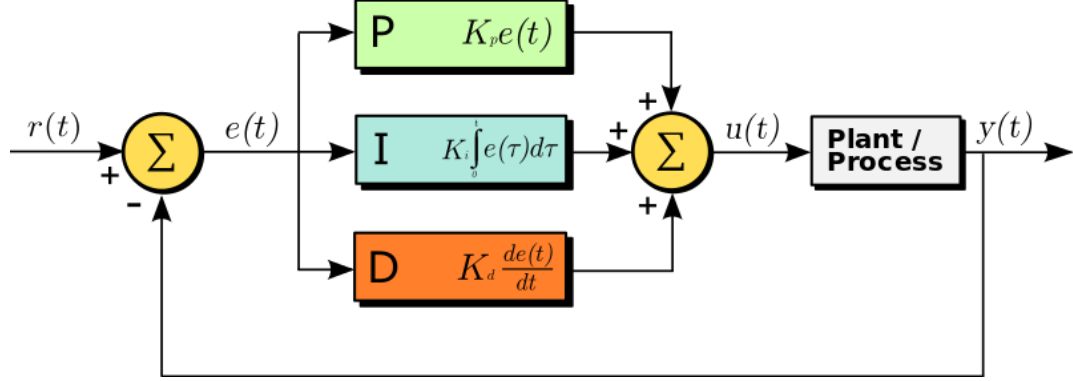


Figure 5.1: PID Controller Feedback System[14]

K_p values.

- **I or Integral gain:** It integrates the calculated error over time and multiplies the integration result by a constant K_i to eliminate the steady-state error, which is the difference between the current and desired output during steady state.
- **D or Derivative gain:** It stabilizes the rate of change of the calculated error over the time by multiplying its derivative value by a constant K_d to avoid high overshoot values.

5.2 Analysis of previous work on axes motion control

The previous work on the Simulink model has created the control loop shown in figure 5.2 [7]. It is a cascaded control loop that receives the motion parameters such as target position, speed, and time generated by the axis flowchart explained in chapter 2.2.2. It also performs the motion control for each axis during each segment in its path along the input curve. It is implemented in a cascaded design where the deviation between the current and target position is fed into PID_1, which generates a speed signal required to reach the target position. The difference between the current speed and the generated speed signal is fed into PID_2, which generates an acceleration signal to reach the

5.2. Analysis of previous work on axes motion control

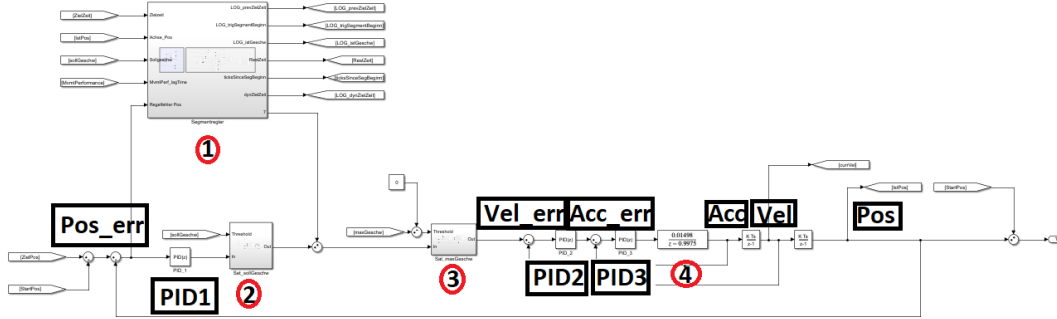


Figure 5.2: Discrete cascaded axis control loop

target speed. The deviation between the current acceleration and the generated signal is fed into PID_3, which generates an acceleration signal to be fed into a discrete transfer function representing the dynamics of the axis. The output of the transfer function is integrated by a discrete-time integral to obtain the axis' actual speed, and the actual speed is integrated to obtain the axis' actual position. That control loop contains four main Simulink blocks numbered in figure 5.2, which will be explained as the following:

1. **Segment controller block:** It is a custom block, which regulates the axis target speed. All agents should reach the end point of the segment at the same instant. This requires that all agents should go with the same target speed determined by dividing the segment length over the the segment time stored on the axis coordination vector. However, the limitations in the axis acceleration makes the axis speed takes time to reach that target speed. Here comes the role of the segment controller. It estimates the deviation in the axis speed and add it to the target speed, so the agent reaches the target position definitely in the target time.
2. **Target speed saturation block:** The position deviation can be very large at the beginning of the segment which makes the speed signal generated by PID_1 exceed the axis target speed. This block saturates this speed to the axis target speed.
3. **Maximum speed saturation block:** Each axis has maximum value of speed at which it can operate. This block saturates the axis speed to the axis maximum speed.

4. **Axis discrete transfer function:** Its a discrete transfer function representing the axis dynamical behaviour.

5.3 Segment controller implementation

The current work was to convert the Simulink blocks shown in chapter 5.2 into C codes, so it can work for the Effectuator. This chapter 5.3 will explain the segment controller embedded implementation while chapter 5.4 will explain the implementation of the rest blocks of the axis control loop. Before going into the implementation details, the following table 5.1 declares the variables manipulated inside the segment controller.

Table 5.1: Segment controller variable declarations

Variable	Data type	Declared value	Description
input variables			
<i>targrt_time</i>	double	0	Segment target time
<i>axis_pos</i>	double	0	Current X2-axis position
<i>axis_pos_state</i>	double	0	Next X2-axis position
<i>target_speed</i>	int	0	Segment target speed
<i>MvmPref_lagTime</i>	int	0	Segment performance lag time
<i>pos_deviation</i>	double	0	X2-axis position error
local variables			
<i>target_pos</i>	double	0	X2-axis target position
<i>prev_axis_pos</i>	double	0	Previous X2-axis postion
<i>sampling_time</i>	const double	0.005	Simulink model sampling time
<i>Segment_controller_numTicks_enable</i>	const int	30	Minimum number of iterations to enable segment controller
<i>Segment_controller_minVel_enable</i>	const int	1	Minimum value of velocity to enable segment controller
<i>sPID_input_k</i>	double	0	Segment controller PID current input
<i>sPID_input_k_1</i>	double	0	Segment controller PID input of one time delay

5.3. Segment controller implementation

<i>sPID_input_k_2</i>	double	0	Segment controller PID input of two time delays
<i>sPID_output_k</i>	double	0	Segment controller PID current output
<i>sPID_output_k_1</i>	double	0	Segment controller PID output of one time delay
<i>sPID_output_k_2</i>	double	0	Segment controller PID output of two time delays
output variables			
<i>prev_target_time</i>	double	0	Segment previous output time
<i>trig_SegmentBeginn</i>	_Bool	false	Trigger Segment beginning
<i>enable_Segcontroller</i>	_Bool	true	Enable Segment controller
<i>actual_speed</i>	int	0	X2-axis current speed
<i>ticksSinceSegmentBeginn</i>	int	0	Current number of iteration since segment started
<i>ticksSinceSegmentBeginn_state</i>	int	0	Next number of iteration since segment started
<i>remaining_time</i>	double	0	Segment remaining time
<i>dyn_target_time</i>	double	0	Dynamic target time (Equivalent of Segment target time and performance lag time)
<i>prev_dyn_target_time</i>	double	0	Previous dynamic target time
<i>sc_output</i>	double	0	Segment controller output

The Effectuator's segment controller is implemented inside a method called **segment_controller()** as the following:

The method **execute_movement()** explained in chapter 4.3.2 detects the beginning of the segment by setting the Boolean variable *trig_SegmentBeginn*. The method **execute_movement()** resets that variable while the control loop is still doing the same segment. The beginning of the segment is detected to update the both *prev_target_time* and *prev_dyn_target_time* of the segment controller and resets *ticksSinceSegmentBeginn*. The segment controller is enabled only if *ticksSinceSegmentBeginn* exceeds *Segment_controller_numTicks_enable* and the absolute value of the segment target speed is less than *Segment_controller_minVel_enable*, which indicates that the agent is too slow

or too fast and it will not reach the target position on the specified target time, so the segment controller output should regulate this deviation. The segment controller calculates the *dyn_target_time* as the segment *target_time* added to the *MvmPref_lagTime*, which indicates the actual target time at which the axis should reach the target position. The segment controller calculates the segment time as the difference between *dyn_target_time*, the *prev_dyn_target_time* and uses this segment time value to detect the segment *remaining_time* by subtracting the elapsed time since the segment has begun represented by the multiplication of *ticksSinceSegmentBeginn* and *sampling_time*. see equation 5.1

$$\begin{aligned} remaining_time = & dyn_target_time \\ & - (ticksSinceSegmentBeginn \cdot sampling_time) \\ & - prev_target_time \end{aligned} \quad (5.1)$$

The segment controller calculates the axis *actual_speed* using the following equation 5.2:

$$actual_speed = \frac{axis_pos - prev_axis_pos}{sampling_time} \quad (5.2)$$

Then, it uses both values of *remaining_time* and *actual_speed* to calculate the error in the position, which it compensates using a PID controller as the following equation 5.3:

$$sPID_input_k = pos_deviation - (actual_speed \cdot remaining_time) \quad (5.3)$$

The output of the PID controller is the output of the segment controller. If the segment controller is disabled, all the PID parameters are reset to zero. Figure 5.3 is a flowchart illustrating the behaviour of the segment controller.

5.4 Axis control loop

As it was mentioned at the beginning of chapter 5.3, the current work implemented the same axis control loop explained in chapter 5.2 as an embedded algorithm to be executed by the Effectuator. It implemented the individual blocks of the control loop and integrated them in one method called **controlLoop()**. Each of the following subsection will explain one of these blocks, **controlLoop()** method and **go_to_pos()** method.

5.4. Axis control loop

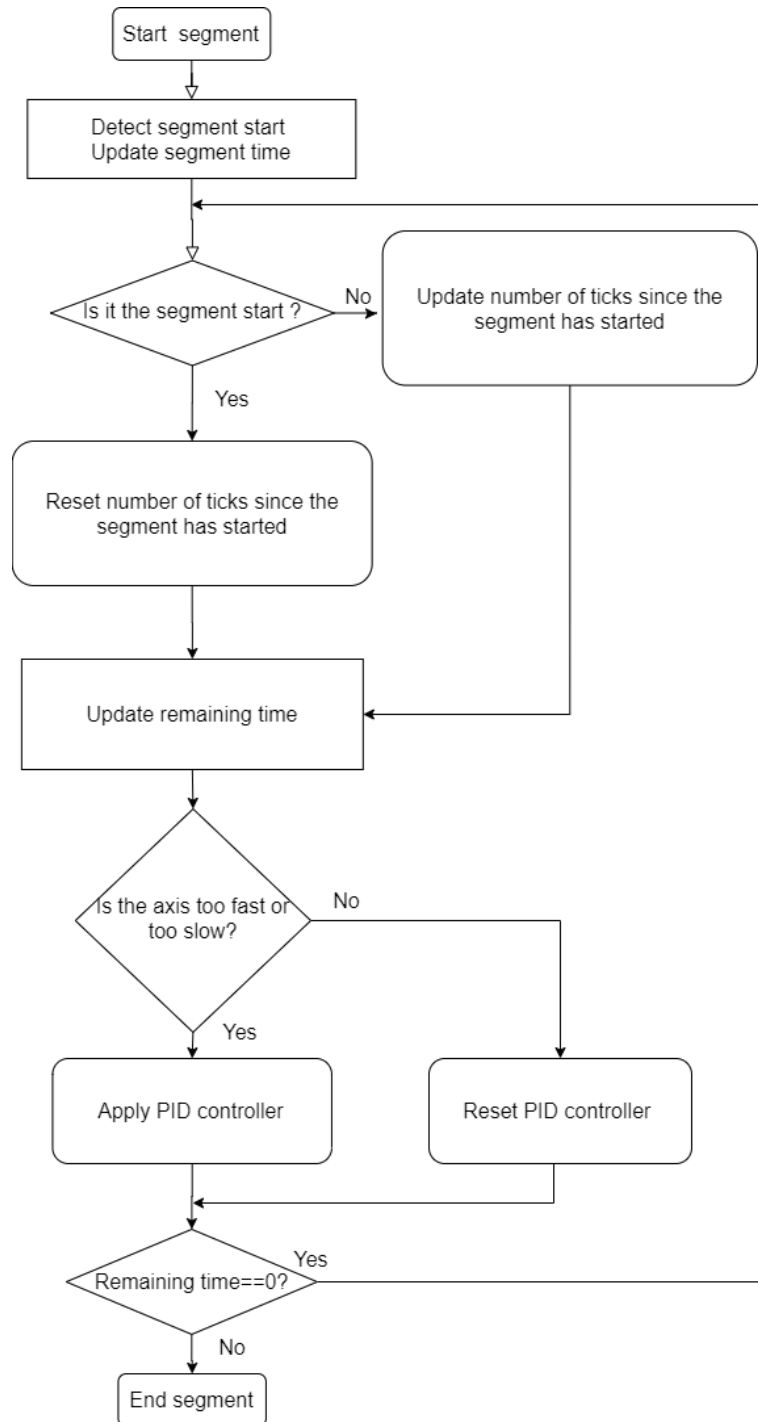


Figure 5.3: Segment controller behaviour

5.4.1 PID controller implementation

The transfer function of the PID controller used by the Simulink model, see equation 5.4 was converted to a difference equation by the mathematical derivation attached in Appendix A.1. This difference equation, see equation 5.5 was used by a function called **PID()** to do the same functionality of the PID controller used by Simulink.

$$PID(z) = P + I \cdot T_s \cdot \frac{1}{z-1} + D \cdot \frac{N}{1 + N \cdot T_s \cdot \frac{1}{z-1}} \quad (5.4)$$

Where P is the proportional gain, I is the integral gain, D is the derivative gain, T_s is the sampling time, N is the filter coefficient.

$$\begin{aligned} u(k) = & (2 - N \cdot T_s)u(k-1) + (N \cdot T_s - 1)u(k-2) + (P + D \cdot N)e(k) \\ & + (P \cdot N \cdot T_s + I \cdot T_s - 2P - 2D \cdot N)e(k-1) \\ & + (P + I \cdot N \cdot T_s^2 - P \cdot N \cdot T_s - I \cdot T_s + D \cdot N)e(k-2) \end{aligned} \quad (5.5)$$

Where $u(k)$ is the current PID output, $u(k-1)$ is the previous PID output, $u(k-2)$ is the former PID input, $e(k)$ is the current PID output, $e(k-1)$ is the previous PID input, $e(k-2)$ is the former PID input. The **PID()** method updates the value of its parameters to be used for the next iteration as the following:

$u(k)$ = the output of the difference equation, $u(k-2)=u(k-1)$, $u(k-1)=u(k)$, $e(k-2)=e(k-1)$, and $e(k-1)=e(k)$.

5.4.2 Saturation block implementation

PID controller has a drawback called integration wind-up. As it was explained in chapter 5.1.1, the integral component of the **PID** integrates the system output error over time. When the system reaches its saturation point, this can lead to the stated wind-up problem. Simulink provides two anti-windup solutions. They are back-calculation and clamping methods, which are built in Simulink **PID** block [15]. These two methods can limit the **PID** block output to a certain fixed range of values. However, in the current case these limits are changing during the Simulation running according to the target speed of each segment. The previous work on the Simulink model [7] created two custom saturation blocks to achieve the dynamical requirements

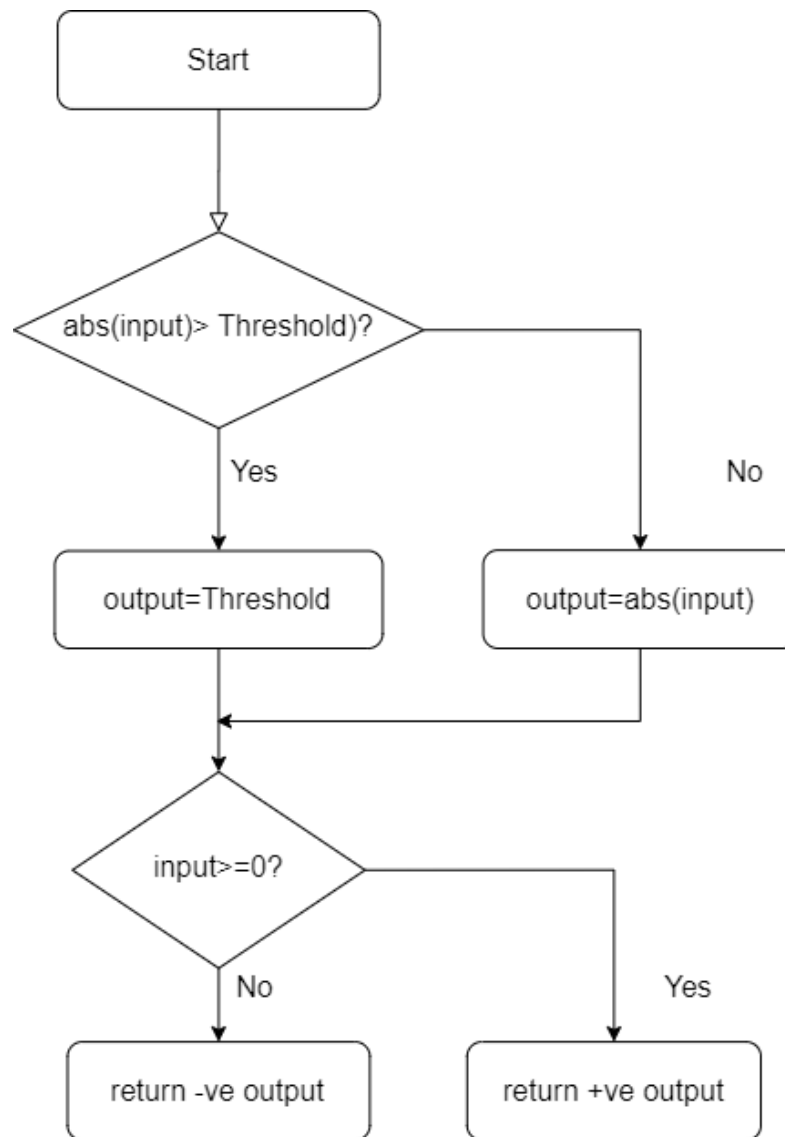


Figure 5.4: `sat.Speed()` function behaviour

of the system as it was explained in chapter 5.2. The current work created **sat_Speed()** function that gives the same behaviour as the two Simulink blocks. Based on the function inputs, it gives one of the two target outputs of the two Simulink blocks. The **sat_Speed()** function was implemented to saturate the value of the speed signal created by PID_1, so it does not exceed either the segment target speed or the axis maximum speed. It consists of two cascaded **If** statements to compare its input signal with the threshold value that should not be exceeded by that input value. The flowchart in figure 5.4 illustrates the the saturation block behaviour.

5.4.3 Discrete transfer function implementation

The discrete transfer function shown in equation 5.6 was converted to a difference equation shown in equation 5.7 by the mathematical derivation attached in Appendix A.2.

$$G(z) = \frac{0.01498}{z - 0.9975} \quad (5.6)$$

$$u(k) = 0.9975u(k-1) + 0.01498e(k-1) \quad (5.7)$$

Where $u(k)$ is the current Discrete transfer function output, $u(k-1)$ is the previous Discrete transfer function output, $e(k)$ is the current Discrete transfer function input, $e(k-1)$ is the previous Discrete transfer function input. The **DiscreteTF()** method updates the value of its parameters to be used for the next iteration as the following:

$u(k)$ = the output of the difference equation, $u(k-1)=u(k)$, and $e(k-1)=e(k)$.

5.4.4 Discrete time integrator implementation

The Discrete time integrator is implemented to integrate the output of the Discrete transfer function, so it obtains the axis current speed. It also integrates this value of the axis current speed to obtain the axis current position. Simply, it executes the two discrete integral equation used by Simulink to perform forward Euler integration as shown in equations 5.8 and 5.9.

$$y(n) = x(n) \quad (5.8)$$

$$x(n+1) = y(n) + T_s \cdot u(n) \quad (5.9)$$

5.4. Axis control loop

Where $y(n)$ is the Discrete time integrator output, $x(n)$ is the Discrete time integrator output state, $u(n)$ is the Discrete time integrator input, and T_s is the sampling time.

5.4.5 Control loop implementation

The current work has implemented the control loop explained in chapter 5.2 to control the motion of X2-axis driven by the Effectuator. The main aim of this control loop is to make the Effectuator reach a target position by dividing its path from its starting position to that target position into several segments. The end point of each segment is also considered as a target position at which it should reach within a target time synchronized with the motion of the other axes after coordination and consensus vectors calculations as it will be explained in chapter 5.5.

The control loop algorithm is implemented inside a method called **control-Loop()**. This method is running inside a **while()** loop inside **go_to_pos()** method that will be explained in chapter 5.4.6. When the time of the current segment is elapsed the **while()** loop is broken and a method called **reset-CLoop()** is called to reset all the control loop parameters, except for the output position, to their initial values so the control loop can work on the new segment. Table 5.2 declares the variables manipulated inside the control loop:

Table 5.2: Control loop variables declaration

Variable	Data type	Declared value	Description
<i>PID1_input_k</i>	double	0	PID 1 current input
<i>PID1_input_k_1</i>	double	0	PID 1 input of one time delay
<i>PID1_input_k_2</i>	double	0	PID 1 input of tow time delay
<i>PID1_output_k</i>	double	0	PID 1 current output
<i>PID1_output_k_1</i>	double	0	PID 1 output of one time delay
<i>PID1_output_k_2</i>	double	0	PID 1 output of two time delays
<i>PID2_input_k</i>	double	0	PID 2 current input
<i>PID2_input_k_1</i>	double	0	PID 2 input of one time delay
<i>PID2_input_k_2</i>	double	0	PID 2 input of tow time delay

<i>PID2_output_k</i>	double	0	PID 2 current output
<i>PID2_output_k_1</i>	double	0	PID 2 output of one time delay
<i>PID2_output_k_2</i>	double	0	PID 2 output of two time delays
<i>PID3_input_k</i>	double	0	PID 3 current input
<i>PID3_input_k_1</i>	double	0	PID 3 input of one time delay
<i>PID3_input_k_2</i>	double	0	PID 3 input of tow time delay
<i>PID3_output_k</i>	double	0	PID 3 current output
<i>PID3_output_k_1</i>	double	0	PID 3 output of one time delay
<i>PID3_output_k_2</i>	double	0	PID 3 output of two time delays
<i>minAllowedVel_Saturation</i>	int const	1	Minimum value of saturated velocity
<i>sat_targetSpeed_output</i>	int	0	Output value of target speed saturation block
<i>sat_maxTargetSpeed_output</i>	int	0	Output value of maximum speed saturation block
<i>DiscreteTF_input_k</i>	double	0	Discrete transfer function current input
<i>DiscreteTF_input_k_1</i>	double	0	Discrete transfer function input of one time delay
<i>DiscreteTF_output_k</i>	double	0	Discrete transfer function current output
<i>DiscreteTF_output_k_1</i>	double	0	Discrete transfer function output of one time delay
<i>currVel</i>	double	0	X2-axis current velocity
<i>currVel.state</i>	double	0	X2-axis next velocity
<i>axis_output</i>	double	0	X2-axis position with respect to its starting point

The control loop calculates the *axis_output* with the following equation 5.10:

$$axis_output = axis_pos + startPos \quad (5.10)$$

Then it calculates the axis *pos_deviation* with the following equation 5.11:

$$pos_deviation = target_pos - startPos - axis_pos \quad (5.11)$$

5.4. Axis control loop

This *pos_deviation* is assigned to be the input of the first PID controller. The output of the first PID controller is a target speed signal that should be saturated by calling the speed saturation block implemented in chapter 5.4.2. The speed saturation block compares this speed signal with its threshold value, which is the maximum of the current segment *target_speed* and the *minAllowedVelSaturation* value. The output of the speed saturation block is assigned to the variable *sat_targetSpeed_output* to be added with the output signal of the segment controller implemented in chapter 5.3 and passed to another call of the speed saturation block to be saturated again. This time, the speed saturation block compares this speed signal with its threshold value, which is the maximum value of the axis speed and assigns the return value to the variable *sat_maxTargetSpeed_output*. Then the control loop starts calculating the three motion parameters (acceleration, speed, position) from the most internal loop to the outer loop of the cascaded structure of the Simulink control loop as it was explained in chapter 5.2. The *PID2_output_k* is assigned to *DiscreteTF_input_k* then the Discrete transfer function implemented in chapter 5.4.3 is called to update its output value *DiscreteTF_output_k* representing the axis new acceleration value. This acceleration value is integrated over the time by calling the Discrete time integrator implemented in chapter 5.4.4 to update the value of the axis speed *currVel*. This value of the axis speed is used to calculate the input of the second PID controller with the following equation 5.12:

$$PID2_input_k = sat_targetSpeed_output - currVel \quad (5.12)$$

The second PID controller is called to generate a target acceleration signal that is used to calculate the third PID controller input with the following equation 5.13:

$$PID3_input_k = PID2_output_k - DiscreteTF_output_k \quad (5.13)$$

Finally, the **segment_controller()**, implemented in chapter 5.3, is called to update its output *sc_output* and the output value of the third PID *PID3_output_k* is assigned to the previous value of the Discrete transfer function to be used in the next iteration of the control loop.

5.4.6 Go to position implementation

The current work has implemented the body of **go_to_pos()** declared by the previous work on the Effectuator [4]. As shown in figure 5.5, the function

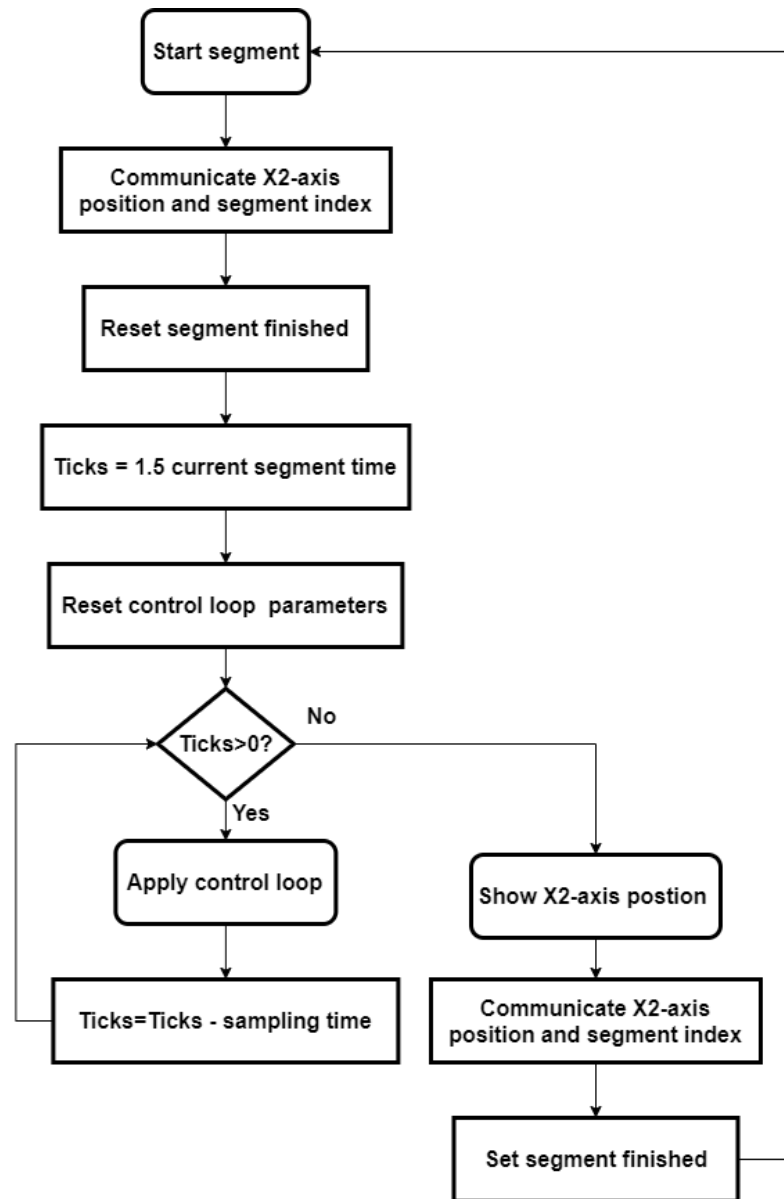


Figure 5.5: `Go_to_pos()` function behaviour

5.5. The Effectuator synchronization with the demonstrator axes

sends a message to the Simulink model to share the following information with the other axes:

- **The current segment index.**
- **The Effectuator current position.**
- **The current segment state:** It indicates if the Effectuator is still doing the current segment or it has finished it.

It resets that current segment state and assigns the value of the current segment time saved in the variable *currStepTimeLength* to a new variable called *ticks*. It calls the method **resetCLoop()** and begins a **while()** loop that calls the control loop explained in chapter 5.4.5. Inside the **while()** loop the value of the variable *ticks* is updated by subtracting the sampling time from it in each iteration. The **while()** loop is broken when the value of that *ticks* variable becomes a negative value. To make sure that the Effectuator has reached the target position an experimental factor of value 1.5 is multiplied by the variable *currStepTimeLength* before assigning its value to the variable *ticks*. Then, the **go_to_pos()** method calls the method **task_showPos()** that displays the Effectuator current position on the **PuTTY** terminal and sends a message to the Simulink model to share the new information with other axes in the same way before the control loop was called but this time it sets the current segment state. The message is prepared by calling a method called **segment_executed()** that collects the required data and returns a variable of type `t_tthing` holding the message contents.

5.5 The Effectuator synchronization with the demonstrator axes

The Effectuator motion was to be synchronized with the motion of the other axes so they execute the same segment at the same time. As it was explained in chapter 4.2, **Sendaxesindex/position** function shares the axes position and segment index with the Effectuator, so it does not exceed the other axes index. At the same time, the Effectuator shares its position and segment indices with the other axes via **go_to_pos()** as it was explained in chapter 5.4.6, so they do not exceed the Effectuator index. The equivalent of these two conditions results in the Effectuator and the demonstrator axes

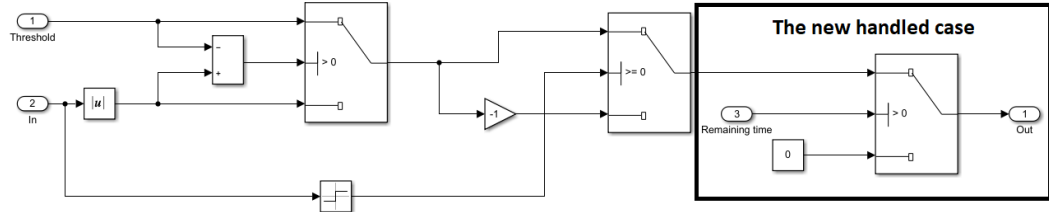


Figure 5.6: The new handled case for maximum speed saturation block

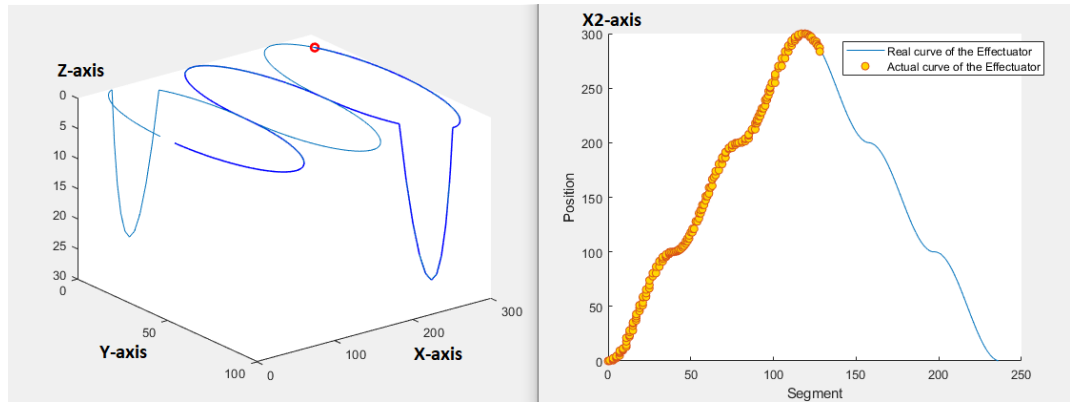


Figure 5.7: Output position of the four axes

going along the path segment by segment. For example, if the Effectuator is paused, the demonstrator axes will hold in position and it goes the same way if the Effectuator is paused. However, a new case was handled for the Simulink **Maximum speed saturation** block described in chapter 5.2 as when the Effectuator is paused and the remaining time of the segment controller becomes a negative value, a negative maximum speed value was passed to the demonstrator axes, so they go back in position. As shown in figure 5.6, this problem was solved by adding another case in **Maximum speed saturation** block, so it passes a minimum speed value if the value of the segment remaining time becomes negative. Figure 5.7 shows the position output of the Effectuator synchronized with the demonstrator axes.

5.6. Segment controller and control loop implementation debugging

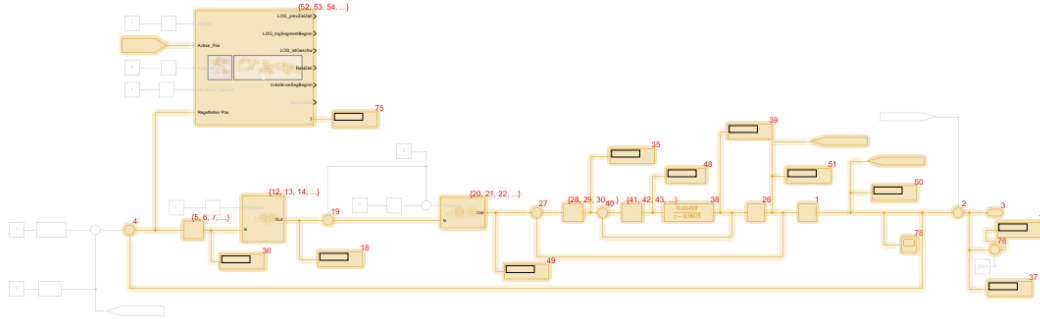


Figure 5.8: controlLoopDepugging Simulink model with display blocks and order execution mode

5.6 Segment controller and control loop implementation debugging

A debugging model was created to test the behaviour of the Effectuator Segment controller and control loop to make sure that they are giving the same results as the Segment controller and control loop of the Simulink model. A separate Simulink model called **controlLoopDepugging** was created for this purpose. The model takes the end point of the first segment as its target position and it is run step by step to compare the value of all signals that appears on the Simulink display blocks, as shown in figure 5.8, with the values of the variables representing these signals in the Effectuator debugging area in the Eclipse software as shown in figure 5.9. Simulink has debugging feature called **Execution order** that was used to track the order of the blocks execution and apply almost the same order of execution in the sequential embedded algorithm of the Effectuator axis control loop explained in chapter 5.4.5, so it behaves similar to the Simulink model. Figure 5.8 shows that order of execution mode where it appears above of each Simulink block.

Integer vs Double calculations

The previous work has recommended that the Effectuator should use integer values in both the communication process and the motion control calculations to save its resources [4]. As it was mentioned in chapter 3.9, the communication process was implemented using integer values in data transmission. However, using integer values in the motion control calculations will cause

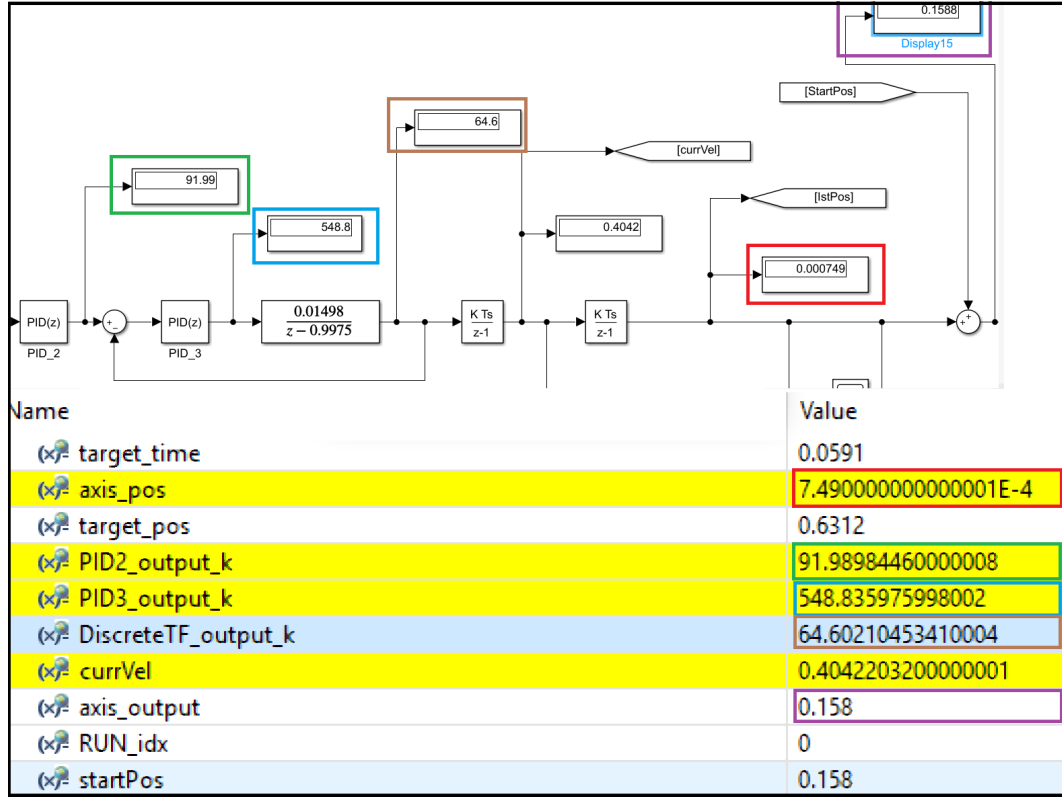


Figure 5.9: Comparing Simulink results with Effectuator results

the following two problems:

- Using integer values means that the minimum value for the system sampling time is 1, which is much bigger than the sampling time of the original time ($T_s = 0.005$), which leads the system output to be instable.
- As shown in figure 5.10, by using Simulink PID tuning method at ($T_s = 1$) the PID controlling parameters are very small values with negative powers, which can be considered as zero values. However, assigning zero values to those parameters leads to disconnecting the control loop and eliminating its output.

To solve these two problems, the motion control calculations had to use double values for time and position parameters of the control loop. The Effectuator used also the same sampling time and **PID** gains used in Simulink model and declared by previous work [7].

5.7. Summary of the chapter

Compensator formula

$$P + I \cdot T_s \frac{1}{z-1} + D \frac{N}{1 + N \cdot T_s \frac{1}{z-1}}$$

Main Initialization Output Saturation Data Types State Attributes

Controller parameters

Source: internal

Proportional (P): 1.99104988547658e-08

Integral (I): 1.93466320446357e-13

Derivative (D): 0.00024047304472957

☒ Use filtered derivative

Filter coefficient (N): 0.136806412492574

Figure 5.10: PID tuning parameter values at ($T_s = 1$)

5.7 Summary of the chapter

The chapter covered the motion control execution implemented and tested for the Effectuator. It illustrated the idea of the PID controller. It illustrated the Simulink motion control loop implemented by the previous work. It explained the current implementation, including all the control loop components (PID, saturation block, discrete transfer function, and segment controller). It also explained the method of debugging the current implementation for all these blocks. Overall, the motion control process was implemented. In following related works, a real motor can be integrated instead of the system equation explained in chapter 5.4.3. It also can perform the motion control commands of the Effectuator.

Chapter 6

Evaluation

6.1 Evaluation of thesis requirements

By looking at the thesis requirements that were illustrated in chapter 1.3, they can be evaluated as the following:

- **UDP communication implementation**

UDP communication was implemented based on the requirements of the previous work as it was explained in chapter 3. The Effectuator can communicate directly with the other agents and vice versa. The message structure used was a general message structure that can be used to transmit data with different content types and lengths. At the same time, it used integer values in data transmission to save the Effectuator resources and increase the number of packets that can be transmitted in a single **UDP** message. Currently, the Effectuator can communicate its coordination vector, consensus vector, X2-axis position, segment index, and motion end flag. The Simulink model can share the path points of the tracked curve, the axes coordination matrix, the axes position, and segment indices. These communicating data between the Effectuator and the Simulink model allowed the Effectuator to be integrated into the demonstrator simulation working area.

Further agents can be integrated into the system without changing the communication structure or building a new one.

- **The Effectuator integration into the original system**

The current work focused on integrating the Effectuator axis with the

other axes of the demonstrator. However, further agents can be integrated into the system with small changes in the current work and the same way in which the Effectuator was integrated. The **UDP** packets length explained in chapter 4.2 should be increased based on the size of the new coordination matrix with coordination vectors of all agents arranged inside it.

- **Motion control execution**

An embedded motion control algorithm was created, tested, and simulated on the Effectuator hardware. It behaves the same as the Simulink motion control algorithms implemented by the previous work. It was created based on the logic of the axis control loop explained in chapter 5.2. The logic of Simulink blocks was translated into C codes, including the blocks used from Simulink libraries or the custom blocks implemented by previous work [7].

6.2 The Effectuator and demonstrator synchronization analysis

The current work implemented the synchronization process with the help of the following parameters:

- **Message data field:** As it was explained in chapter 3.2.2, this part of the message structure represents the content type of the messages sent and received between the Effectuator and the Simulink model. It also indicates that both of them entered a new step in their process. For example, the Effectuator keeps checking the change in the message field of the incoming UDP packets to start a new step in its process and handle the incoming packet if it carries path points, coordination matrix, or the segment indices of the other axes. The Simulink model also uses this message data field to recognize the different messages received from the Effectuator if it is a coordination vector, consensus vector, or position and segment index of the Effectuator. It also uses this message data field as a transition condition between different states in the Effectuator (Axis-X2) state flow explained in chapter 4.3.1.
- **Message to the Effectuator:** The current work added this new output parameter of the **Agents/State flow** block explained in chapter

2.2.2. It was added to synchronize the Effectuator (Axis-X2) state flow with the **Communication stack** explained in chapter 4.2, so the messages are sent to the Effectuator not only based on its current state but also based on the other axes state.

- **Communicating axes position and segment indices:** As it was explained in chapter 4.2, **Send_axes_index/position** communicates the position and segment indices of the demonstrator axes. On the Effectuator side **go_to_pos()** communicates the position and segment index of X2-axis. With these communicating data, both of the Effectuator and the Simulink model keep tracking the position and segment index of each other. An extra condition was added to the state flow charts of the Simulink axes that was it was explained in chapter 2.2.2, not to exceed the segment indices of the Effectuator. A condition was added to the **execute_movement()** function that was explained in chapter 4.3.2 not to exceed the indices of the Simulink axes. The result of these two conditions was that both of the Effectuator and the demonstrator axes are synchronized with each other and going along the path segment by segment as it was shown in figure 5.7.

6.3 The requirements for the real motor and non-simulated demonstrator

The following list will explain some expected requirements to integrate a real motor to the real hardware of the demonstrator, so the motor can be driven with the motion control algorithm developed on the Effectuator.

- **Obtaining a new transfer function:** To integrate the real motor, the difference equation used in the current implementation and explained in chapter 5.4.4 should be replaced by the difference equation of the motor. This can be done by obtaining the transfer function of the motor either from its model or by the practical method explained in [16] if the motor model is unknown. After that, this continuous transfer function should be converted to its discrete-time equivalent using **c2d** MATLAB command. After that, the obtained discrete transfer function should be converted to a difference equation to be used in the Effectuator C code.

6.3. The requirements for the real motor and non-simulated demonstrator

- **Joystick Ctrl block updating:** The **Joystick Ctrl** clarified in chapter 2.2.2 should be updated, so it can interact with both of the Effectuator and the demonstrator. The coordination data of the Effectuator (coordination vector, consensus vector, and execution vector) should also be updated and communicated during the motion execution not only before it.
- **Motor wiring diagram:** A wiring diagram should be created for the new motor, so it is easy to integrate its hardware connections with the hardware connections of the original setup. This wiring diagram should cover the motor power source connections, the logic connections from the Effectuator to the motor driver, and the Ethernet connection between the Effectuator and the Simulation **PC**.

Chapter 7

Summary and Outlook

7.1 Summary

In this thesis, the Effectuator, which is an embedded NIOS board, was integrated with the demonstrator hardware setup existing at **ISW**. This setup represents the implementation of the communication and control motion concepts developed throughout the work on **DEVEKOS** research project.

At first, the thesis analyzed the previous work on the project. It analyzed the Simulink models implemented to simulate and drive the motion of the three transitional axes X-axis, Y-axis, and Z-axis of the demonstrator. It also analyzed the previous work on the Effectuator board, including its integration concept, the communication structure, the message structure, and motion coordination calculations.

The thesis started with communication implementation based on the recommendation and requirements of the previous work. It implemented a **UDP** communication structure with two different algorithms. An embedded algorithm for the Effectuator and another Simulink algorithm for the demonstrator. Both algorithms were based on a unique message structure, so all agents can share their data regardless if they are derived by the embedded board or by **PC** simulations. That resulted in a successful communication process between the Effectuator and the demonstrator.

7.1. Summary

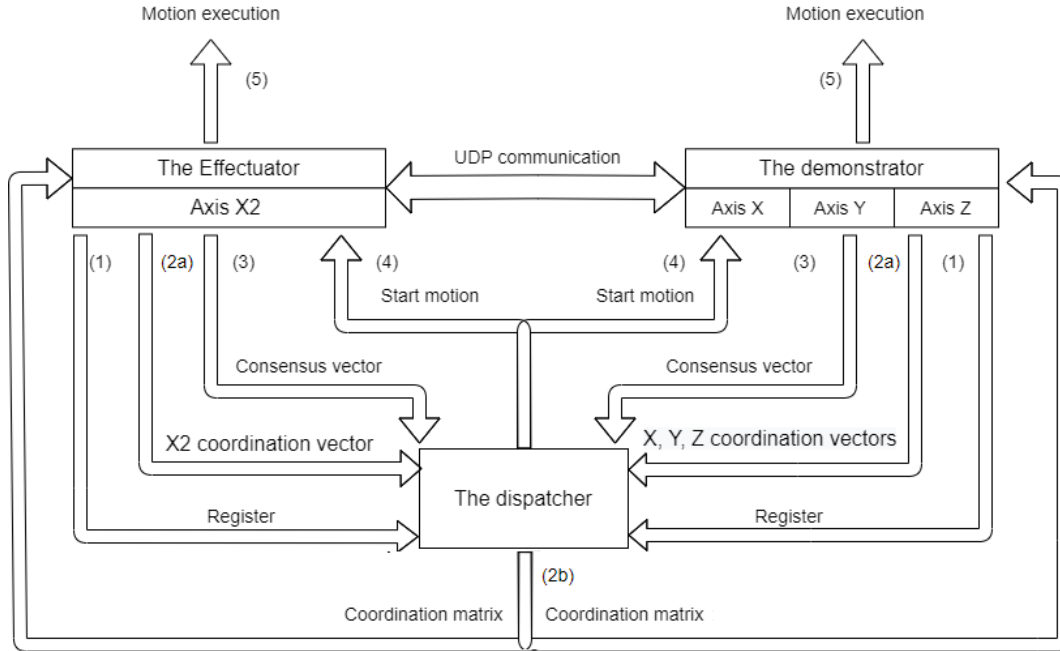


Figure 7.1: The entire process of the new system

Further, the Effectuator was inserted in the working area of the demonstrator axes so all axes can do and share their calculations before and during the motion execution process.

An embedded motion control algorithm was implemented based on the Simulink motion control models of the demonstrator axes. This algorithm was implemented and tested to control the motion of a new axis called X2-axis that should be driven by the Effectuator in following related work. Figure 7.1 illustrates the entire process of the new system of the integrated Effectuator and the demonstrator as it was explained in chapter 2.3.

7.2 Outlook

For further work, the new agent X2-axis can be actuated by a real motor. A brushless DC motor of type DB42S03 can be connected to the Effectuator. The future work should replace the current transfer function illustrated in equation 5.6 with the transfer function of the new motor and obtain a new difference equation with the same way explained in chapter 5.4.3 and same mathematical derivation explained in Appendix A.2.

Future work can also integrate new agents with new embedded FPGA/NIOS **CPUs**. In this case, the message structure indicated in chapter 3.2.2 should be extended with a new field called, for example, *agent_number*. This new field represents which agent is communicating and to which agent should the message be sent. Besides, the **Communication stack** implemented in chapter 4.2 should be extended, so it handles packets for more than one agent.

Bibliography

- [1] Caren Dripke. International bachelor thesis 2020 topic proposal. Technical report, Institute for Control Technology of Machine Tools and Manufacturing Equipment, Stuttgart, Germany, 2019.
- [2] ISW. Devekos (durchgängiges engineering für sichere, verteilte und kommunizierende mehrkomponentensysteme) (german title), (integrated engineering for secure, distributed and communicating multi-component systems) (english title). institute for control technology of machine tools and manufacturing equipment. <https://www.isw.uni-stuttgart.de/forschung/steuerungsalgorithmen/devekos/>, February 2017 (Publish date), June 2020 (Access date).
- [3] Caren Dripke, Sonja Laicher, and Alexander Verl. Axis-in-the-loop simulation: a concept to transfer verification and validation of distributed interpolation algorithms from simulation to interaction with commercial motion control hardware. In *Proceedings of International Conference on Computers Industrial Engineering (CIE 48)*, 2018.
- [4] Pirmin Stanglmeier. Application case study for decentralized interpolation: Integration of commercial drive components and prototypical devekos axis control components. Master’s thesis, Institute for Control Engineering of Machine Tools and Manufacturing Units (ISW), Stuttgart, Germany, 2020.
- [5] Caren Dripke, Zhongyi Ye, and Alexander Verl. Synchronization of a distributed interpolation application via cross-coupled control. In *IEEE International Conference on Engineering, Technology and Innovation (ICE/ITMC)*. IEEE, 2019.

- [6] Caren Dripke, Daniel Schoebel, and Alexander Verl. Synchronization of motion-controlled axes with coordination vector and decentralized segment controller. In *IEEE International Workshop on Advanced Motion Control (AMC2020)*. IEEE, 2020.
- [7] Daniel Schoebel. Development of a decentralized control concept for distributed interpolation with autonomous agents. Master's thesis, Institute for Control Engineering of Machine Tools and Manufacturing Units (ISW), Stuttgart, Germany, May 2019.
- [8] Caren Dripke, Ben Schneider, Mihai Dragan, Alois Zoitl, and Alexander Verl. Concept of distributed interpolation for skill-based manufacturing with real-time communication. In Thorsten Schüppstuhl, Kirsten Tracht, and Jörg Franke, editors, *Tagungsband des 3. Kongresses Montage Handhabung Industrieroboter*. Springer Vieweg, 2018.
- [9] Caren Dripke and Alexander Verl. Challenges in distributed interpolation with multi-component systems in future-oriented manufacturing units. In *Proceedings of International Conference on Computers Industrial Engineering (CIE 48)*, 2017.
- [10] IDbox. Industrial communications. <https://idboxrt.com/en/industrial-communications/>, January 2018 (Publish date), June 2020 (Access date).
- [11] Nisha Jiju. What's the difference between ip/tcp udp? <https://www.colocationamerica.com/blog/tcp-ip-vs-udp>, December 2018 (Publish date), June 2020 (Access date).
- [12] Lewis Van Winkle. *Hands-On Network Programming with C: Learn socket programming in C and write secure and optimized network code*. Packt Publishing Ltd, 2019.
- [13] NonGNU. lwip: Overview. https://www.nongnu.org/lwip/2_0_x/index.html, October 2017 (Publish date), June 2020 (Access date).
- [14] Zakariyya Hassan Abdullahi, Bashir Ahmed Danzomo, and Zainab Suleiman Abdullahi. Design and simulation of a pid controller for motion control systems. In *IOP Conference Series: Materials Science and Engineering*, volume 344, 2018.

Bibliography

- [15] MathWorks. Anti-windup control using a pid controller. <https://de.mathworks.com/help/simulink/slref/anti-windup-control-using-a-pid-controller.html>, July 2005 (Publish date) , August 2020 (Access date).
- [16] Changle Xiang, Xiaoliang Wang, Yue Ma, and Bin Xu. Practical modeling and comprehensive system identification of a bldc motor. *Mathematical Problems in Engineering*, 2015.

Appendix A

Mathematical derivation for difference equations

A.1 Discrete PID controller

Starting with Simulink PID equation :

$$PID(z) = P + I \cdot T_s \cdot \frac{1}{z-1} + D \cdot \frac{N}{1 + N \cdot T_s \cdot \frac{1}{z-1}} \quad (\text{A.1})$$

Multiplying the numerator and denominator of the third term by (z-1) :

$$PID(z) = P + I \cdot T_s \cdot \frac{1}{z-1} + \frac{D \cdot N(z-1)}{(z-1) + N \cdot T_s} \quad (\text{A.2})$$

Applying common denominator :

$$PID(z) = \frac{P(z-1)(z-1 + N \cdot T_s) + I \cdot T_s \cdot (z-1 + N \cdot T_s) + D \cdot N(z-1)^2}{(z-1)(z-1 + N \cdot T_s)} \quad (\text{A.3})$$

Solving the parentheses :

$$PID(z) = \frac{P(z-1)^2 + P(z-1) \cdot N \cdot T_s + I \cdot T_s \cdot (z-1) + I \cdot N \cdot T_s^2 + D \cdot N(z-1)^2}{(z-1)^2 + (z-1)N \cdot T_s} \quad (\text{A.4})$$

Separating z factor :

$$PID(z) = \frac{(P + D \cdot N)z^2 + (-2P + P \cdot N \cdot T_s + I \cdot T_s - 2D \cdot N)z + (P - P \cdot N \cdot T_s - I \cdot T_s + I \cdot N \cdot T_s^2 + D \cdot N)}{z^2 + (-2 + N \cdot T_s)z + (1 - N \cdot T_s)} \quad (A.5)$$

Multiplying the numerator and denominator by z^{-2} :

$$PID(z) = \frac{(P + D \cdot N) + (-2P + P \cdot N \cdot T_s + I \cdot T_s - 2D \cdot N)z^{-1} + (P - P \cdot N \cdot T_s - I \cdot T_s + I \cdot N \cdot T_s^2 + D \cdot N)z^{-2}}{1 + (-2 + N \cdot T_s)z^{-1} + (1 - N \cdot T_s)z^{-2}} \quad (A.6)$$

Substituting by the PID transfer function as an output/input :

$$PID(z) = \frac{U(z)}{E(z)} \quad (A.7)$$

Applying cross multiplication :

$$\begin{aligned} U(z) + (-2 + N \cdot T_s)z^{-1}U(z) + (1 - N \cdot T_s)z^{-2}U(z) &= (P + D \cdot N)E(z) \\ &+ (-2P + P \cdot N \cdot T_s + I \cdot T_s - 2D \cdot N)z^{-1}E(z) \\ &+ (P - P \cdot N \cdot T_s - I \cdot T_s + I \cdot N \cdot T_s^2 + D \cdot N)z^{-2}E(z) \end{aligned} \quad (A.8)$$

Applying z inverse :

$$\begin{aligned} u(k) + (-2 + N \cdot T_s)u(k-1) + (1 - N \cdot T_s)u(k-2) &= (P + D \cdot N)e(k) \\ &+ (-2P + P \cdot N \cdot T_s + I \cdot T_s - 2D \cdot N)e(k-1) \\ &+ (P - P \cdot N \cdot T_s - I \cdot T_s + I \cdot N \cdot T_s^2 + D \cdot N)e(k-2) \end{aligned} \quad (A.9)$$

Separating u(k) at the left side :

$$\begin{aligned} u(k) &= (2 - N \cdot T_s)u(k-1) + (N \cdot T_s - 1)u(k-2) + (P + D \cdot N)e(k) \\ &+ (-2P + P \cdot N \cdot T_s + I \cdot T_s - 2D \cdot N)e(k-1) \\ &+ (P - P \cdot N \cdot T_s - I \cdot T_s + I \cdot N \cdot T_s^2 + D \cdot N)e(k-2) \end{aligned} \quad (A.10)$$

A.2 Discrete axis transfer equation

starting with Simulink discrete transfer equation :

$$G(z) = \frac{0.01498}{z - 0.9975} \quad (\text{A.11})$$

Multiplying the numerator and denominator by z^{-1} :

$$G(z) = \frac{0.01498z^{-1}}{1 - 0.9975z^{-1}} \quad (\text{A.12})$$

Substituting by the discrete transfer function as an output/input :

$$G(z) = \frac{U(z)}{E(z)} \quad (\text{A.13})$$

Applying cross multiplication :

$$U(z) - 0.9975z^{-1}U(z) = 0.01498z^{-1}E(z) \quad (\text{A.14})$$

Applying z inverse :

$$u(k) - 0.9975u(k-1) = 0.01498e(k-1) \quad (\text{A.15})$$

Separating $u(k)$ at the left side :

$$u(k) = 0.9975u(k-1) + 0.01498e(k-1) \quad (\text{A.16})$$